



2-Layer LiDAR-Inertial SLAM Architecture

ID :

Lần ban hành : 01

Ngày ban hành :

Số trang :

Mức độ bảo mật :

Biên soạn	Kiểm tra	Phê Duyệt
Nguyễn Quang Thiện	Lê Tấn Đạt	Trần Quang Khôi

Table of Contents

I.	Executive Summary	4
II.	System Architecture Overview	6
III.	Layer 1: Fusion Engine.....	8
	3.1 Kinematic Model	9
	3.2 Iterated Error-State Kalman Filter (IEKF) on Manifold	10
	3.2.1 Forward Propagation.....	11
	3.2.2 Measurement Model	11
	3.2.3 Iterated Update.....	12
	3.3 Motion Undistortion.....	14
	3.4 ikd-Tree: Incremental K-D Tree	15
	3.4.1 Node Structure	17
	3.4.2 kNN Search.....	17
	3.4.3 Point Insertion with On-tree Downsampling	18
	3.4.4 Box-wise Delete with Lazy Labels	19
	3.4.5 Re-balancing & Parallel Re-building.....	19
	3.5 Direct Point Registration.....	21
	3.6 Local Mapping Management	22
IV.	Layer 2: VoxMap Manager.....	24
	4.1 Spatial Hashing	29



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 2/53

4.2 History Buffer & Sequential Voxel Removal	30
4.3 Pool Allocator	30
4.4 3D DDA Raycasting	31
4.5 Log-Odds Occupancy Model	33
4.6 Processing Timeout.....	35
4.7 Frame-Age Eviction.....	35
4.8 Incremental Map Broadcast (Optional)	36
V. Complexity Summary	37
VI. Benchmark Results	38
6.1 Layer 1 Performance.....	41
6.2 Layer 2 Performance.....	42
6.3 Middleware bandwidth & Incremental Broadcast	43
VII. Architecture Strengths	44
7.1 Deterministic Real-Time.....	44
7.2 Pre-allocated Memory Model	45
7.3 Multi-Sensor Platform	45
7.4 Bounded-Memory Footprint	45
7.5 Unidirectional Data Flow	46
7.6 Scalability for Navigation Stack	46
VIII. Limitations & Future Improvements	47
8.1 No Loop Closure (Design Tradeoff).....	47

8.2 No Degeneracy Detection (Layer 1)	47
8.3 Online Extrinsic Estimation (Layer 1)	47
8.4 Single-Resolution Voxel Grid (Layer 2).....	48
8.5 No Semantic Information (Layer 2).....	48
8.6 Ghost Voxel Outside FoV (Layer 2).....	49
IX. Parameter Configuration.....	50
X. Conclusion	52

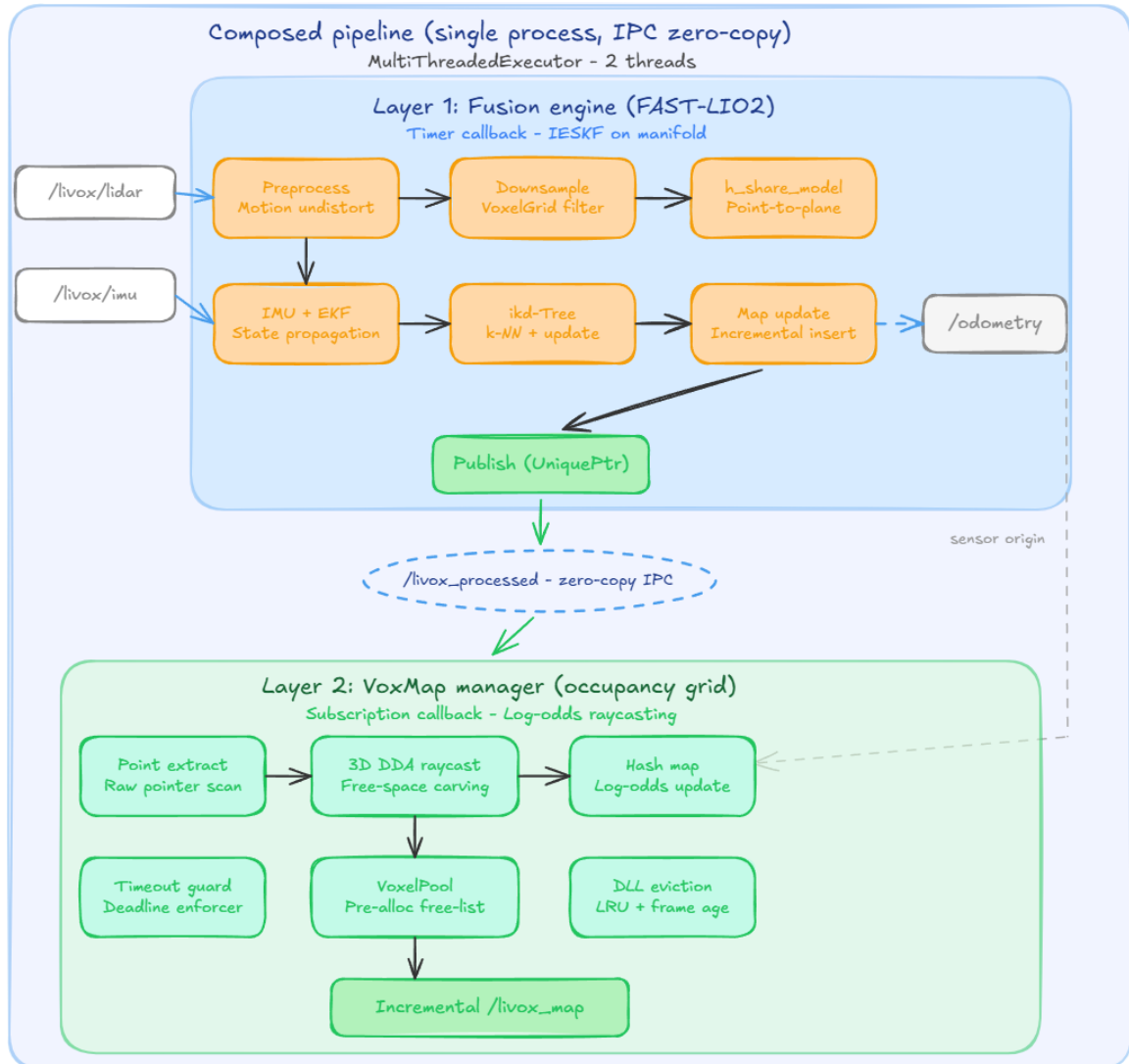


2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 4/53

I. Executive Summary

- **Mục tiêu:** Xây dựng hệ thống bản đồ thời gian thực (real-time mapping) cho robot tự hành, xử lý dữ liệu point cloud 3D từ các loại cảm biến LiDAR khác nhau, spinning LiDAR như Velodyne, Ouster, solid-state LiDAR như Livox AVIA, MID-360,... kết hợp cảm biến quán tính IMU. Hệ thống phải đảm bảo xử lý hoàn tất trong thời gian xác định 100ms/frame (10Hz) trên phần cứng nhúng. Kiến trúc không phụ thuộc vào một loại cảm biến LiDAR cụ thể, cho phép hoán đổi cảm biến mà không cần thay đổi kiến trúc cốt lõi.
- **Cốt lõi:** Cung cấp **bản đồ vật cản** xung quanh UAV với latency xác định để planner luôn có dữ liệu cập nhật mỗi frame và tối ưu cho việc truy xuất, hoạch định đường đi.
- **Kiến trúc:** *Hệ thống SLAM LiDAR-Quán tính 2 Lớp* xử lý tuần tự.
 - Lớp 1 (Fusion Engine) triển khai thuật toán FAST-LIO2, thực hiện ước lượng trạng thái 6-DOF bằng bộ lọc IEKF ghép cặp chặt LiDAR-IMU, sử dụng cấu trúc dữ liệu ikd-Tree cho bản đồ điểm dày đặc và đăng ký trực tiếp điểm thô không qua trích xuất đặc trưng.
 - Lớp 2 (VoxMap Manager) lấy cảm hứng từ hệ thống voxel mapping với cấu trúc hash table, quản lý voxel bằng ưu tiên không-thời gian, nhận đám mây điểm đã đăng ký và cập nhật bản đồ voxel chiếm dụng với mô hình xác suất log-odds.
- **Cơ sở lý thuyết:** Kiến trúc 2 lớp này kết hợp hai nền tảng nghiên cứu.
 - Lớp 1 dựa trên **FAST-LIO2 (Xu et al., 2022)** với bộ lọc Kalman lắp ghép cặp chặt hiệu suất cao và ikd-Tree hỗ trợ bản đồ tăng dần.
 - Lớp 2 lấy cảm hứng từ **Voxel Mapping System (La et al., 2024)** sử dụng hash table và quản lý voxel động, được tái thiết kế với pool allocator, LRU và frame-age eviction để đạt hiệu năng thời gian thực xác định trên phần cứng nhúng.



Tổng quan Kiến trúc Hệ thống SLAM LiDAR-Quản tính 2 Lớp

- **Kết quả:** Tổng thời gian xử lý trung bình của cả **2 Layer** đạt ngưỡng **~30%** trong khoảng thời gian **100ms** được định (**10Hz**). Đạt hiệu quả **~99%** trong các điều kiện môi trường rộng lớn và môi trường có nhiều vật thể động.

Mean end-to-end latency $\approx L1 (15\text{ ms}) + L2 (15\text{ ms}) \approx 30\text{ ms/frame}$

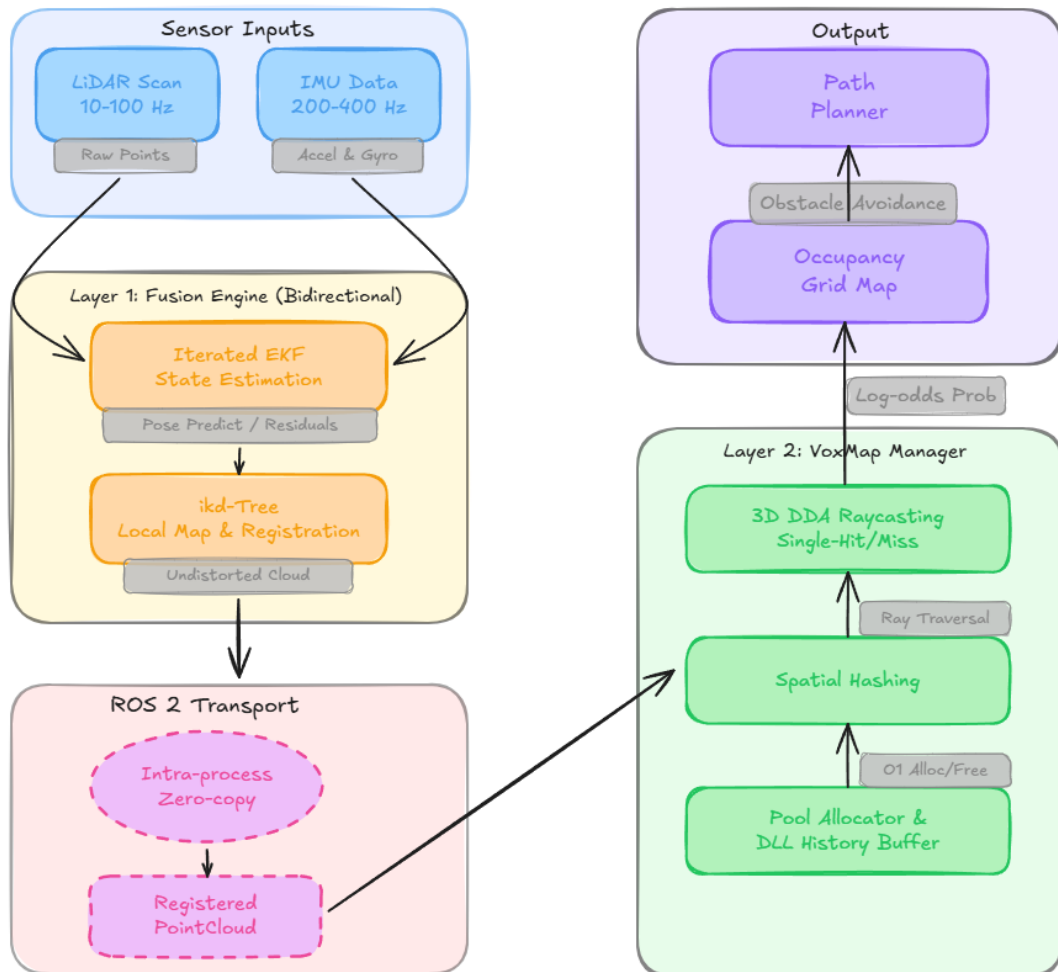
II. System Architecture Overview

Hệ thống sử dụng kiến trúc pipeline 2 lớp với luồng dữ liệu 1 chiều (Unidirectional Data flow). Thiết kế này cho phép tối ưu và kiểm thử độc lập từng lớp mà không ảnh hưởng lẫn nhau.

Tính linh động: Layer 1 không phụ thuộc vào bất kỳ loại cảm biến LiDAR cụ thể nào nhờ phương pháp đăng ký trực tiếp (direct registration) không cần trích xuất đặc trưng, hỗ trợ xử lý thông qua module tiền xử lý (preprocess) có thể cấu hình.

Luồng dữ liệu:

Sensors Data (LiDAR + IMU) → [Layer 1: IEKF + ikd-Tree] → Registered PointCloud2
→ [Layer 2: VoxMap] → Occupancy Grid → Navigation Planner.



Tổng quan Luồng xử lý dữ liệu

Thông số	Layer 1 (Fusion Engine)	Layer 2 (VoxMap Manager)
Chức năng	Ước lượng trạng thái 6-DOF, bù trừ chuyển động, đăng ký scan-to-map, cập nhật bản đồ điểm	Bản đồ chiếm dụng voxel, mô hình xác suất log-odds, quản lý bộ nhớ với LRU
Thuật toán chính	IEKF trên đa tạp $SO(3) \times R^{15} \times SO(3) \times R^3$, point-to-plane ICP, back-propagation	Spatial hashing $O(1)$, log-odds Bayesian update, lazy temporal decay
Cấu trúc dữ liệu	ikd-Tree, incremental KD-tree với parallel re-balancing	unordered_map + Doubly-Linked List + Pool Allocator
Complexity	$O(m_{raw} + m \cdot \log(N) + K \cdot \dim^2 \cdot m)$	$O(1)$ hash + $O(1)$ DLL
Input	PointCloud2 (LiDAR) + IMU	/livox_processed (PointCloud2)
Output	/livox_processed, /odometry	/livox_map (incremental)



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 8/53

III. Layer 1: Fusion Engine

Một hệ thống LiDAR-Inertial Odometry ghép cặp chặt (tightly-coupled) đã được chứng minh hoạt động ổn định với nhiều loại cảm biến LiDAR. Điểm chính của FAST-LIO2 là phương pháp trực tiếp (direct method) đăng ký điểm thô vào bản đồ mà không cần trích xuất đặc trưng, giúp hệ thống tự động thích ứng với các mẫu quét (scanning pattern) khác nhau mà không cần chỉnh sửa tham số trích xuất.

Luồng xử lý tổng thể của Layer 1 mỗi khi nhận dữ liệu từ sensor LiDAR như sau:

(1) **Forward Propagation:** Giữa hai lần scan LiDAR liên tiếp, IMU đã sinh nhiều mẫu đo. Mỗi mẫu IMU được dùng để lan truyền trạng thái và hiệp phương sai về phía trước theo phương trình động học. Giúp dự đoán pose tại thời điểm scan mới đến, kèm theo mức độ không chắc chắn dựa trên hiệp phương sai.

(2) **Motion Undistortion:** LiDAR quét tuần tự và mỗi điểm được đo tại một thời điểm khác nhau khi chuyển động. Dựa vào quỹ đạo IMU đã lưu, timestamp riêng của từng điểm, hệ thống nội suy chính xác tư thế UAV tại thời điểm đo điểm đó, rồi chiếu tất cả điểm về cùng một tư thế tham chiếu (thời điểm cuối scan). Kết quả là một lần scan “sạch” không còn méo chuyển động, sẵn sàng cho đối sánh bản đồ.

(3) **Scan-to-Map Matching:** Mỗi điểm trong lần scan đã bù trừ được chiếu từ hệ tọa độ LiDAR sang hệ toàn cục qua pose dự đoán. Sau đó, hệ thống tìm kiếm điểm lân cận gần nhất (k-NN) trong bản đồ ikd-Tree và fit một mặt phẳng cục bộ qua các điểm lân cận đó. Residual tính khoảng cách từ điểm đã chiếu đến mặt phẳng (point-to-plane distance). Đo lường mức độ sai lệch giữa pose dự đoán và thực tế. Tập hợp residual của tất cả điểm trong scan tạo thành đầu vào cho bước update.

(4) **Iterated Update:** Sử dụng tập residual và covariance từ trên, bộ lọc IEKF giải bài toán tối ưu, tìm lượng hiệu chỉnh pose sao cho tổng residual nhỏ nhất, đồng thời không đi quá xa khỏi dự đoán IMU (cân bằng giữa tin IMU và tin LiDAR qua Kalman gain). Quá trình được lặp nhiều vòng vì bài toán phi tuyến khi mỗi lần pose thay đổi, điểm chiếu vào bản đồ cũng thay đổi, mặt phẳng lân cận có thể khác, residual thay đổi, cần tính lại. Lặp cho đến khi hiệu chỉnh đủ nhỏ (hội tụ).



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 9/53

(5) **Map Update:** Sau khi pose đã hội tụ, scan được chiếu lại vào hệ toàn cục với pose đã hiệu chỉnh và chèn vào ikd-Tree. Trong quá trình chèn, để đảm bảo mật độ điểm đồng đều thì kiến trúc sử dụng on-Tree Downsampling (mỗi voxel chỉ giữ một điểm). Nếu bản đồ cục bộ chạm biên, vùng bản đồ dịch chuyển và các điểm ngoài vùng mới bị box-wise delete. Scan đã đăng ký được gửi xuống Layer 2.

3.1 Kinematic Model

Trạng thái hệ thống được định nghĩa trên manifold (đa tạp) với tổng cộng 24 chiều tự do.

$$M = SO(3) \times R^{15} \times SO(3) \times R^3, \dim(M) = 24$$

Manifold này là tích trực tiếp của các không gian thành phần:

- $SO(3)$ thứ nhất biểu diễn attitude (hướng) của IMU trong hệ tọa độ toàn cục tương ứng 3 bậc tự do quay.
- R^{15} chứa 5 vector, mỗi vector 3 chiều, lần lượt là vị trí, vận tốc, sai lệch gyroscope, sai lệch accelerometer, và vector trọng lực.
- $SO(3) \times R^3$ (3 chiều dịch chuyển) biểu diễn extrinsic parameters (tham số ngoại) giữa LiDAR và IMU, gồm độ quay tương đối và độ dịch tương đối giữa hai cảm biến trên body (thân UAV).

Kinematic equation (phương trình động học) mô tả cách trạng thái tiến hoá theo đầu vào IMU. Đạo hàm hướng IMU phụ thuộc vào vận tốc góc đo được, khi đã trừ sai lệch gyroscope và nhiễu. Đạo hàm vị trí bằng vận tốc tuyến tính. Đạo hàm vận tốc bằng gia tốc đo được, khi xoay về hệ toàn cục, trừ sai lệch accelerometer, và cộng trọng lực. Các bias được mô hình hoá như random walk process (quá trình bước ngẫu nhiên). Tham số ngoại và trọng lực có đạo hàm bằng 0 vì chúng không đổi theo thời gian.

Phương trình liên tục trên được rời rạc hoá tại chu kỳ lấy mẫu IMU (Δt) bằng toán tử boxplus ($\boxplus$) trên manifold.

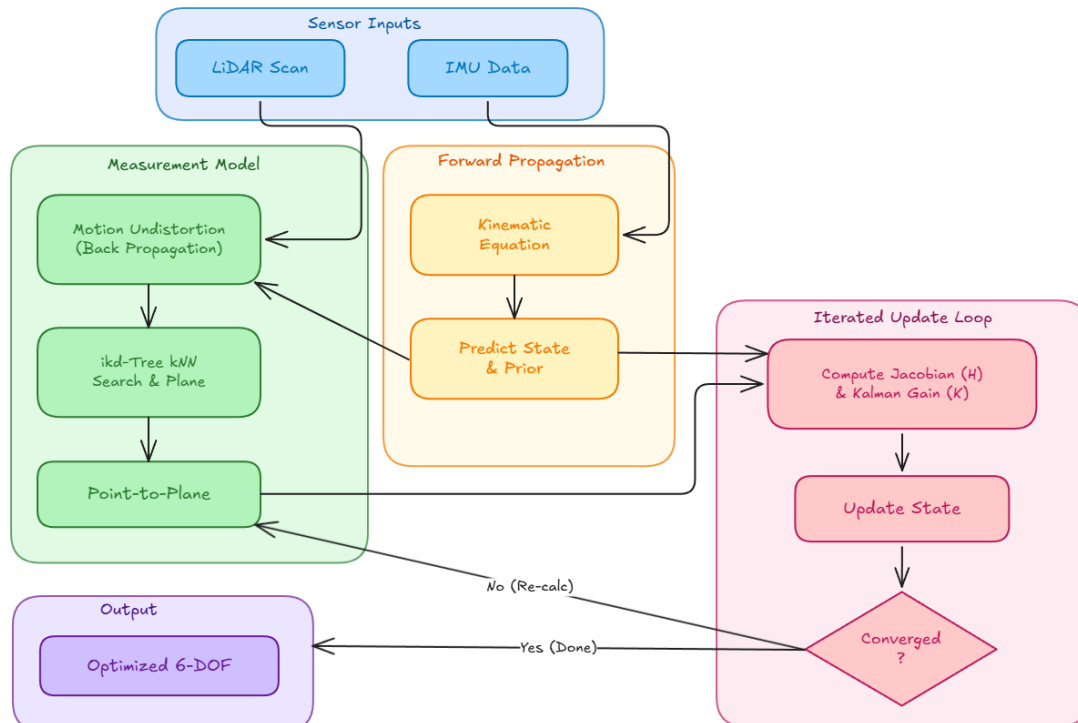
$$x(i+1) = x(i) \boxplus (\Delta t \cdot f(x(i), u(i), w(i)))$$

Trong đó x là vector trạng thái 24 chiều, $u = [\omega_m, a_m]^T$ là đầu vào IMU, vận tốc góc và gia tốc đo được, và $w = [n\omega, n_a, nb\omega, nb_a]^T$ là process noise (nhiều quá trình). Toán tử boxplus đảm bảo phần quay sử dụng phép nhân ma trận quay qua ánh xạ Exp từ đại số Lie sang nhóm Lie, còn phần Euclid sử dụng phép cộng thông thường. Nhờ vậy, kết quả luôn nằm đúng trên manifold mà không cần chuẩn hoá lại quaternion sau mỗi bước.

- Phép quay 3D thuộc đối tượng trên nhóm Lie $SO(3)$, có hình học cong và đóng kín. Khi biểu diễn phép quay bằng góc Euler gặp gimbal lock, đôi khi lại mất bậc tự do tại một số tư thế đặc biệt. Khi biểu diễn phép quay bằng quaternion thì cần chuẩn hoá lại ($\|q\| = 1$) sau mỗi cập nhật, gây sai số tích lũy. Định nghĩa trạng thái trên manifold loại bỏ cả hai vấn đề, cho phép hệ thống hoạt động ổn định ngay cả khi robot quay tới $1000^\circ/s$.

3.2 Iterated Error-State Kalman Filter (IEKF) on Manifold

IEKF là thành phần trung tâm thực hiện sensor fusion (hợp nhất cảm biến) giữa IMU và LiDAR. Bộ lọc hoạt động theo hai nhịp tương ứng với hai luồng dữ liệu cảm biến có tần số khác nhau.



Bộ lọc IEKF (Iterated Error-State Kalman Filter)

Tần số IMU thực hiện propagation (lan truyền thuận), mỗi mẫu IMU đến, bộ lọc dự đoán trạng thái và covariance (hiệp phương sai), ma trận đặc trưng cho độ không chắc chắn về phía trước. Tần số LiDAR thực hiện iterated update, tương ứng với mỗi lần LiDAR scan đến, bộ lọc hiệu chỉnh dự đoán dựa trên đối sánh Scan-to-Map. Sự chênh lệch tần số này là đặc điểm của *tightly-coupled fusion* (hợp nhất ghép cặp chặt), khi mà IMU giữ cho ước lượng trạng thái liên tục, không bị “đóng băng” giữa các lần scan LiDAR.

3.2.1 Forward Propagation

Mỗi khi nhận một mẫu IMU mới (gồm vận tốc góc ω_m và gia tốc a_m), bộ lọc thực hiện hai việc song song.

Thứ nhất, cập nhật trạng thái bằng cách áp dụng phương trình động học đã rời rạc hoá, đặt nhiễu quá trình $w = 0$, giả sử không có nhiễu để dự đoán và ảnh hưởng của nhiễu sẽ được phản ánh thông qua hiệp phương sai.

Thứ hai, lan truyền covariance (hiệp phương sai) của error-state (sai số trạng thái) theo công thức sau.

$$\hat{P}(i+1) = F\tilde{X} \cdot \hat{P}(i) \cdot F\tilde{X}^T + Fw \cdot Q \cdot Fw^T$$

Trong đó, $F\tilde{X}$ là Jacobian (ma trận đạo hàm riêng) của phương trình động học theo error-state, mô tả cách sai số trạng thái hiện tại lan truyền sang bước tiếp theo. Fw là Jacobian theo nhiễu, mô tả mức độ nhiễu IMU ảnh hưởng đến từng thành phần trạng thái. Và Q là IMU noise covariance matrix (ma trận hiệp phương sai nhiễu IMU), đặc trưng cho độ ồn cố hữu của cảm biến.

- Sau mỗi mẫu IMU là một cặp giá trị, ước lượng trạng thái mới và hiệp phương sai tương ứng. Hiệp phương sai sẽ tăng dần theo thời gian nếu không có phép đo LiDAR để hiệu chỉnh. Covariance tăng đơn điệu trong giai đoạn propagation thuận, phản ánh sự tích lũy không chắc chắn khi chỉ dựa vào dead-reckoning (ước tính) IMU. Chỉ đến lần scan LiDAR tiếp theo đến, bước update mới “kéo” hiệp phương sai về giá trị nhỏ hơn.

3.2.2 Measurement Model

Lần scan LiDAR mới đến, hệ thống sẽ xây dựng measurement model (mô hình đo lường) để biểu diễn mối quan hệ giữa trạng thái ước lượng và dữ liệu LiDAR. Mô hình này chính là cầu nối cho phép bộ lọc so sánh “UAV nghĩ đang ở đâu” với “LiDAR đang thấy gì”.

Mỗi điểm p trong scan (đo trong hệ toạ độ cục bộ của LiDAR) được chiếu sang hệ global frame (toạ độ toàn cục) thông qua chuỗi biến đổi toạ độ.

$$p_{\text{world}} = R_{\text{imu}} \times (R_{\text{extrinsic}} \times p + t_{\text{extrinsic}}) + t_{\text{imu}}$$

Trong đó R_{imu} và t_{imu} là hướng và vị trí của IMU trong hệ toàn cục, lấy từ trạng thái ước lượng hiện tại, $R_{\text{extrinsic}}$ và $t_{\text{extrinsic}}$ là phép quay và dịch chuyển từ hệ LiDAR sang hệ IMU (tham số ngoại). Nếu pose ước lượng chính xác, điểm chiếu p_{world} phải nằm trên bề mặt của một vật thể đã có trong bản đồ. Đây là ràng buộc hình học cốt lõi.

Để kiểm tra ràng buộc này, hệ thống xác định local plane (mặt phẳng cục bộ) tương ứng với mỗi điểm bằng hai bước.

- Một là dùng kNN (k-Nearest Neighbors, tìm k điểm lân cận gần nhất) trên ikd-Tree.
- Hai là fit (khớp) một mặt phẳng qua k điểm đó bằng Householder QR decomposition, thu được normal vector (pháp tuyến) u và một điểm q nằm trên mặt phẳng.

Từ đó, residual (phần dư đo lường, đại lượng đo mức sai lệch giữa ước lượng và quan sát) cho mỗi điểm định nghĩa là khoảng cách có dấu từ điểm chiếu đến mặt phẳng tương ứng.

$$z = u^T \cdot (p_{\text{world}} - q)$$

Nếu pose ước lượng đúng, điểm chiếu sẽ nằm sát mặt phẳng và $\text{residual} \approx 0$. Nếu pose sai, điểm chiếu lệch khỏi mặt phẳng và residual lớn. Mô hình này được gọi là implicit measurement model (mô hình đo lường ẩn) và được biểu diễn phép đo dưới dạng ràng buộc là $0 = h(x, n)$ thay vì dạng tường minh $y = h(x) + n$. Các điểm có residual vượt ngưỡng, tức các outlier (điểm ngoại lai), sẽ bị loại khỏi quá trình update để tránh làm nhiễu kết quả.

3.2.3 Iterated Update

Từ tập residual của tất cả m điểm trong scan kết hợp với hiệp phương sai từ bước propagation, cần sử dụng Maximum A-Posteriori (cực đại hoá xác suất hậu nghiệm), tức tìm trạng thái cân bằng tốt nhất giữa hai nguồn thông tin.

Nguồn thứ nhất là prior distribution (phân phối tiên nghiệm) từ IMU, đặc trưng bởi trạng thái dự đoán $\hat{x}_k$ và hiệp phương sai $\hat{P}_k$, thể hiện “IMU nghĩ robot đang ở đâu với độ tin cậy bao nhiêu”. Nguồn thứ hai là measurement distribution (phân phối đo lường) từ LiDAR, đặc trưng bởi tập residual z và ma trận nhiễu đo lường R , thể hiện “LiDAR nhận xét rằng pose phải dịch chuyển bao nhiêu để các điểm khớp với bản đồ”. Và Kalman gain K sẽ là phần quyết định mức cân bằng giữa hai nguồn.

$$K = (H^T R^{-1} H + P^{-1})^{-1} H^T R^{-1}$$

Trong đó H là Jacobian của mô hình đo lường, mô tả residual thay đổi thế nào khi pose thay đổi, R là measurement noise covariance (ma trận hiệp phương sai nhiễu đo lường), và P là covariance đã biến đổi từ propagation.

- Điểm đột phá nằm ở công thức Kalman gain trên. Thông thường, Kalman gain yêu cầu nghịch đảo ma trận kích thước $m \times m$ với m là số điểm đo (có thể hàng nghìn), tốn $O(m^3)$. Công thức mới tổng hợp thông tin từ tất cả điểm vào một ma trận 24×24 thông qua tích $H^T R^{-1} H$, rồi nghịch đảo ma trận 24×24 đó một lần duy nhất.

- Kết quả là độ phức tạp giảm từ $O(m^3)$ xuống $O(\text{dim}^2 \times m)$ với $\text{dim} = 24$ là số chiều trạng thái, giảm hàng trăm đến hàng nghìn lần thời gian tính toán.

Bài toán Scan-to-Map matching về bản chất là phi tuyến, khi pose thay đổi, điểm chiếu vào bản đồ cũng thay đổi, mặt phẳng lân cận có thể khác, Jacobian H và residual z thay đổi theo. Một lần update không đủ chính xác khi sai lệch ban đầu lớn. Do đó, quá trình update được lặp nhiều vòng, mỗi vòng thực hiện bốn bước.

(a) Tính lại residual z và Jacobian H tại pose hiện tại, vì pose vừa thay đổi từ vòng trước nên mặt phẳng lân cận có thể đã khác.

(b) Tính Kalman gain K và lượng hiệu chỉnh pose tương ứng.

- (c) Cập nhật pose bằng toán tử boxplus ($\boxplus$), đảm bảo kết quả nằm trên đa tạp.
- (d) Kiểm tra hội tụ (convergence): nếu lượng hiệu chỉnh nhỏ hơn ngưỡng ϵ , dừng lặp.

Bộ lọc EKF thông thường chỉ thực hiện một vòng 3 bước rồi dừng lại, không đủ chính xác khi tuyến tính hoá xa điểm tối ưu. IEKF lặp cho đến khi hội tụ, đạt độ chính xác cao hơn đáng kể với chi phí tính toán tăng không nhiều.

3.3 Motion Undistortion

LiDAR không chụp ảnh tức thời như camera mà quét tuần tự, mỗi điểm được đo tại một thời điểm khác nhau. Nếu coi tất cả các điểm như được đo từ cùng một tư thế, scan sẽ bị méo và bản đồ tạo ra sẽ bị nhoè, đặc biệt nghiêm trọng khi robot chuyển động nhanh.

Giải quyết vấn đề này bằng phương pháp back-propagation (lan truyền ngược). Khi mà phần forward propagation trước đã tính và lưu quỹ đạo IMU, tức chuỗi các tư thế trung gian giữa hai lần scan liên tiếp. Mỗi điểm LiDAR mang theo timestamp riêng và dựa vào đó mà hệ thống nội suy chính xác tư thế tại thời điểm đo điểm đó từ quỹ đạo IMU, rồi chiếu điểm về tư thế tham chiếu cuối scan. ***Linear Interpolation with Kinematic Propagation*** (*Nội suy tuyến tính và Lan truyền động học*) là phương pháp được sử dụng.

- (a) **Interpolation weight (Tỷ lệ thời gian)**: hệ thống tính toán trọng số nội suy α dựa trên khoảng cách thời gian từ điểm LiDAR t_i đến các mẫu IMU.

$$\alpha = (t_i - t_k) / (t_{k+1} - t_k)$$

- (b) **IMU interpolation (Nội suy vận tốc và gia tốc)**: Vận tốc góc ω_i và gia tốc tuyến tính a_i được nội suy tại thời điểm t_i .

$$\omega_i = (1 - \alpha) \omega_k + \alpha \omega_{k+1}$$

$$a_i = (1 - \alpha) a_k + \alpha a_{k+1}$$

- (c) **Kinematic Propagation (Lan truyền động học)**: sau khi có ω_i và a_i , trạng thái của UAV tại t_i (bao gồm Pose R_i , t_i) được tính toán thông qua tích phân Euler. Cuối cùng,

điểm LiDAR p_i được chiếu ngược về mốc thời gian chốt khung (scan-end time) t_{end} để khử Motion Skew (méo chuyển động).

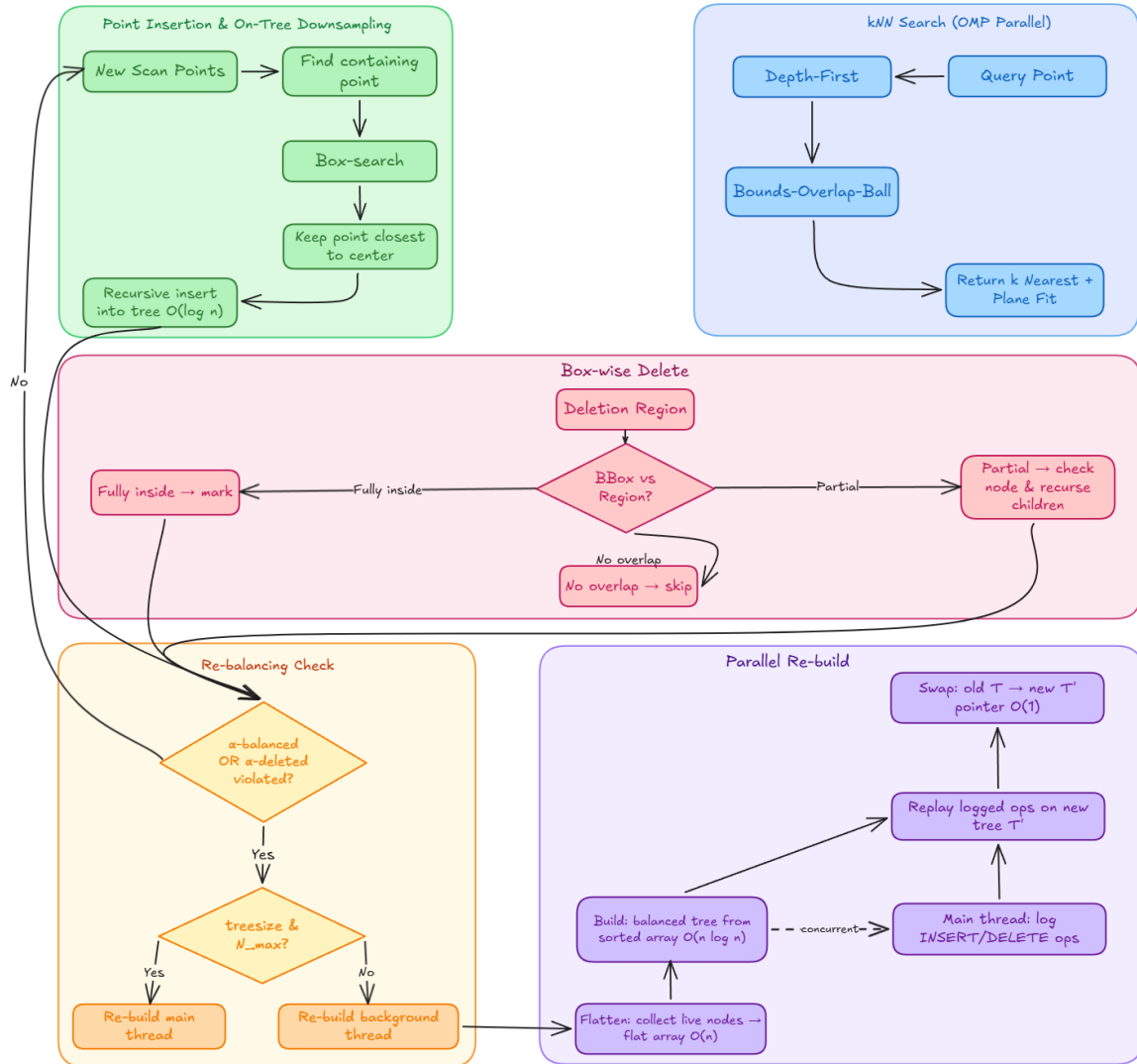
$$P_{comp} = R_L^{I^{-1}} (R_{end}^{-1} [R_i (R_L^I p_i + t_L^I) + t_i - t_{end}] - t_L^I)$$

Trong đó p_i là điểm LiDAR bị méo tại thời điểm t_i , P_{comp} là điểm đã khử méo. R_L^I, t_L^I là extrinsic calibration (ma trận ngoại lai). R_i, t_i là Pose của IMU tại thời điểm t_i . Và R_{end}, t_{end} là Pose của IMU tại thời điểm chốt khung scan.

Phương pháp này xét chính xác quỹ đạo phi tuyến của từng điểm riêng lẻ. Điều này tạo ra sự khác biệt và độ chính xác vượt trội so với các phương pháp xấp xỉ truyền thống vốn thường sai lầm khi giả định chuyển động của robot giữa hai lần quét LiDAR là một chuyển động thẳng đều.

3.4 ikd-Tree: Incremental K-D Tree

ikd-Tree là cấu trúc dữ liệu lưu 3D point cloud map (bản đồ điểm), phục vụ hai chức năng:



ikd-Tree Operations & Parallel Re-building

(a) Truy vấn **kNN**, **k-Nearest Neighbors** (tìm k điểm lân cận gần nhất) phục vụ cho bước measurement model. Mỗi điểm trong lần scan cần tìm k điểm gần nhất trong bản đồ để xác định mặt phẳng đối sánh. Một lần scan có hàng nghìn điểm, mỗi điểm cần một lần kNN, cần bỏ ra thời gian xử lý nhiều nhất.

(b) Cập nhật bản đồ: sau khi pose hội tụ, các điểm trong scan được chiếu sang hệ toàn cục và chèn vào bản đồ, đồng thời các điểm cũ nằm ngoài vùng bản đồ cục bộ cần được xoá.

Vấn đề với các thư viện k-d tree truyền thống (ANN, nanoflann, FLANN) được thiết kế như một static data structure (cấu trúc dữ liệu tĩnh), cây được xây dựng một lần từ tập điểm

cố định rồi chỉ phục vụ truy vấn. Khi bản đồ thay đổi (thêm điểm mới, xóa điểm cũ), toàn bộ cây phải được xây lại từ đầu với chi phí $O(n \log n)$. Với bản đồ hàng trăm nghìn điểm, đây là bottleneck (điểm nghẽn) nghiêm trọng.

- *ikd-Tree* giải quyết bằng cách hỗ trợ incremental update (cập nhật tăng dần): chèn điểm, xóa điểm, và cân bằng lại cây đều có độ phức tạp $O(\log n)$, giảm thời gian mapping xuống đáng kể so với phiên bản kd-Tree.

3.4.1 Node Structure

Mỗi node lưu: tọa độ điểm 3D, con trái/phải, trục chia (x/y/z, luân phiên theo chiều sâu), thống kê cây con cho treesize và invalidnum là số node đã xóa, cờ lazy delete cho node đơn, treedeleted cho cả cây con, và khối bao, bounding box chứa tất cả điểm trong cây con, dùng để pruning (cắt tỉa) trong kNN và box-delete. Khác với nhiều triển khai chỉ lưu điểm tại lá, *ikd-Tree* lưu ở cả node nội và lá, giúp chèn động và cân bằng lại hiệu quả hơn.

3.4.2 kNN Search

kNN search là thao tác được gọi nhiều nhất trong toàn bộ pipeline. Mỗi điểm trong scan cần đúng một lần kNN để tìm mặt phẳng đối sánh cho bước measurement model. Hệ thống duy trì một priority queue (hàng đợi ưu tiên) q chứa k điểm gần nhất tìm được cho đến thời điểm hiện tại, cùng khoảng cách tương ứng đến điểm truy vấn. Bắt đầu từ node gốc, giải thuật depth-first traversal (duyệt cây theo chiều sâu). Tại mỗi node, trước khi đi sâu vào cây con, giải thuật thực hiện kiểm tra bounds-overlap-ball test: tính khoảng cách nhỏ nhất $d_{\min}$ từ điểm truy vấn đến bounding box (khối bao) của cây con.

Nếu $d_{\min}$ lớn hơn hoặc bằng khoảng cách tới điểm xa nhất hiện có trong priority queue q , toàn bộ cây con được pruning (cắt tỉa), không thể có điểm nào trong cây con đó gần hơn k điểm đã tìm được, nên không cần duyệt thêm.

Thêm vào đó áp dụng ranged search (tìm kiếm có giới hạn bán kính), chỉ các điểm nằm trong bán kính cho trước quanh điểm truy vấn mới được xem là inlier (điểm hợp lệ) và được dùng trong bước state estimation. Bán kính này tạo thành ngưỡng cắt tỉa bổ sung, giúp loại bỏ sớm hơn các nhánh cây nằm xa điểm truy vấn.



Độ phức tạp kỳ vọng của một lần kNN search trên ikd-Tree là $O(\log n)$. Chiều cao tối đa của cây được đảm bảo không vượt quá $\log_1/\alpha(n)$ nhờ cơ chế cân bằng, và số lần backtracking (quay lui) tỉ lệ với một hằng số không phụ thuộc kích thước cây.

Về việc xử lý đa luồng: kNN search chỉ **read-only** (không thay đổi cấu trúc hay dữ liệu), nhiều thread có thể truy vấn đồng thời trên cùng cây mà không cần khoá lock hay đồng bộ hoá synchronization, không gây ra race condition (tranh chấp dữ liệu). Việc giải quyết đa luồng chạy nhiều truy vấn đồng thời giúp giảm tổng thời gian cho toàn bộ lần scan đi một cách đáng kể.

3.4.3 Point Insertion with On-tree Downsampling

Sau khi IEKF hội tụ, các điểm trong scan được chiếu sang hệ toàn cục với pose đã hiệu chỉnh rồi chèn vào ikd-Tree. Tuy nhiên, nếu chèn thẳng mọi điểm, bản đồ sẽ đặc dần không đều, vùng UAV đi qua nhiều lần sẽ chồng chất hàng nghìn điểm chỉ cách nhau vài mm, khiến lãng phí bộ nhớ và làm chậm kNN search. Các hệ thống truyền thống giải quyết bằng voxel grid filter (lọc lưới voxel) như một bước tiền xử lý riêng trước khi chèn vào cây. Việc tích hợp downsampling (giảm mẫu) vào ikd-Tree ngay trong quá trình chèn sẽ loại bỏ bước tiền xử lý riêng biệt.

Không gian 3D sẽ được chia thành lưới đều các khối lập phương cube có cạnh bằng filter size map (độ phân giải bản đồ). Mỗi khối chỉ giữ đúng một điểm đại diện, điểm gần tâm khối nhất. Khi điểm mới p đến, hệ thống thực hiện bốn bước tuần tự.

- (a) Xác định khối chứa p dựa trên toạ độ.
- (b) Tìm tất cả điểm hiện có trong khối đó bằng box-wise search (tìm kiếm theo vùng) trên ikd-Tree.
- (c) So sánh khoảng cách từ từng điểm (bao gồm cả p mới) đến tâm khối, chọn điểm gần tâm nhất làm điểm đại diện.
- (d) Xoá các điểm còn lại trong khối, chèn điểm đại diện vào cây nếu nó chưa có.

Bước chèn cuối cùng vào cây sẽ thực hiện đệ quy khi từ node gốc, so sánh tọa độ điểm mới với node hiện tại theo trục chia axis, đi sang cây con trái nếu nhỏ hơn, cây con phải nếu lớn hơn hoặc bằng, cho đến khi gặp vị trí trống thì tạo node mới. Sau mỗi lần chèn, các thuộc tính (treesize, range, invalidnum) của các node trên đường đi được cập nhật, và tiêu chí cân bằng được kiểm tra. Độ phức tạp tổng thể của quá trình chèn kèm downsampling là $O(\log n)$.

3.4.4 Box-wise Delete with Lazy Labels

Khi bản đồ cục bộ dịch chuyển, hệ thống cần xóa tất cả điểm nằm ngoài vùng bản đồ mới. Thay vì tìm và gỡ vật lý từng điểm khỏi cây (tốn kém vì phải tái cấu trúc cây), ikd-Tree sử dụng chiến lược lazy delete (xóa lười): chỉ đánh dấu cờ deleted cho node đơn, hoặc treedeleted cho toàn bộ cây con, mà không thực sự gỡ node khỏi cấu trúc. Các node đã đánh dấu sẽ được dọn dẹp hoàn toàn khi cây con tương ứng được xây lại trong quá trình tiếp theo.

Giải thuật Box-wise Delete nhận đầu vào là một hình hộp cuboid định nghĩa vùng cần xóa, rồi duyệt cây đệ quy sử dụng bounding box (khối bao) của mỗi cây con để cắt tia. Và sẽ có ba trường hợp xử lý.

- (a) Khối bao không giao vùng xóa và bỏ qua toàn bộ cây con, không cần đi sâu hơn.
- (b) Khối bao nằm hoàn toàn trong vùng xóa, đánh dấu cả cây con ngay với chỉ một thao tác duy nhất.
- (c) Giao một phần sẽ kiểm tra node hiện tại, xóa nếu nằm trong rồi đệ quy tiếp vào cây con trái và cây con phải.

Nhờ cơ chế cắt tia ở 2 trường hợp đầu, phần lớn các nhánh cây được bỏ qua hoặc xóa hàng loạt mà không cần duyệt từng node. Độ phức tạp trung bình là $O(\log n)$, phụ thuộc vào tỉ lệ kích thước vùng xóa so với toàn bộ không gian bản đồ.

3.4.5 Re-balancing & Parallel Re-building

Việc chèn và xóa liên tục gây ra hai vấn đề tích lũy ảnh hưởng đến hiệu năng kNN search.

(a) **Tree imbalance:** chèn nhiều điểm vào cùng một vùng không gian khiến nhánh sâu nhánh nông. Giải thuật kNN search phải đi sâu hơn mức cần thiết, duyệt nhiều node thừa, làm chậm đáng kể.

(b) **Dead node accumulation:** việc sử dụng là Box-wise Delete nên việc xóa chỉ bật cờ đánh dấu chứ không thực sự gỡ node khỏi cây. Các node chết vẫn nằm nguyên trong cấu trúc cây, vẫn chiếm bộ nhớ, và rõ ràng là vẫn phải duyệt qua chúng.

ikd-Tree giám sát cả hai vấn đề qua hai tiêu chí tại mỗi cây con:

Tiêu chí α -balanced (cân bằng kích thước): kiểm tra tỉ lệ kích thước giữa cây con trái và cây con phải. Nếu kích thước một nhánh vượt quá $\alpha_{\text{bal}} \times (\text{tổng kích thước} - 1)$ với $\alpha_{\text{bal}} \in (0.5, 1)$, cây con bị coi là mất cân bằng. Tiêu chí này tương tự criterion của scapegoat tree, đảm bảo chiều cao tối đa của cây không vượt quá $\log_2/\alpha(n)$.

Tiêu chí α -deleted (tỉ lệ node chết): kiểm tra tỉ lệ số node đã xóa (invalidnum) so với tổng node (treesize). Nếu $\text{invalidnum} \geq \alpha_{\text{del}} \times \text{treesize}$ với $\alpha_{\text{del}} \in (0, 1)$, cây con chứa quá nhiều node chết và cần được dọn dẹp.

Khi một trong hai tiêu chí bị vi phạm, cây con tương ứng được re-build (xây lại) qua ba giai đoạn tuần tự.

(a) **Flatten:** duyệt in-order toàn bộ cây con, thu thập tất cả node còn sống vào một mảng phẳng (flat array), bỏ qua hoàn toàn các node đã đánh dấu xóa. Sau bước này, mảng chỉ chứa điểm hợp lệ. Độ phức tạp $O(n)$ với n là kích thước cây con.

(b) **Build:** từ mảng đã sắp xếp, xây perfectly balanced tree (cây cân bằng hoàn hảo) bằng đệ quy khi chọn phần tử giữa làm gốc, nửa trái thành cây con trái, nửa phải thành cây con phải. Kết quả là cây mới không còn node chết, chiều cao tối thiểu. Độ phức tạp $O(n \log n)$.

(c) **Swap:** gán lại con trỏ gốc cây con cũ sang cây con mới, giải phóng bộ nhớ cây cũ. Chỉ tốn $O(1)$ vì chỉ thay đổi một con trỏ.

Chiến lược phân chia luồng: với cây con nhỏ ($\text{treesize} < N_{\text{max}}$), cả ba giai đoạn chạy trong luồng chính vì chi phí không đáng kể. Với cây con lớn, chỉ Flatten $O(n)$ và Swap

$O(1)$ chạy trong luồng chính, thực hiện build $O(n \log n)$ chạy trong luồng nền. Luồng chính tiếp tục xử lý scan bình thường không bị chặn.

Xử lý đồng bộ giữa hai luồng: trong lúc luồng nền build cây mới T' từ mảng đã được flatten, luồng chính vẫn nhận scan mới và cần chèn/xóa điểm. Sẽ có thời điểm dữ liệu được phân vào vùng không gian của cây đang được build lại.

- Cần khởi tạo một operation logger (danh sách ghi thao tác) trong thời gian build, luồng chính vẫn được thực hiện và dùng kNNsearch trên cây cũ T , đồng thời ghi lại thao tác INSERT, BOX_DELETE và tham số điểm hoặc vùng xóa vào một mảng động vector (dynamic array) với độ phức tạp tối ưu khi chỉ cần thêm tuần tự mà không phải thêm/xóa giữa mảng, liên kết.

Khi luồng nền hoàn tất cây mới T' , luồng chính sẽ dừng lại trong một khoảng thời gian ngắn (không đáng kể):

(a) **Replay** phát lại và thực thi toàn bộ các thao tác đã ghi trong operation logger lên T' theo đúng thứ tự ghi, đảm bảo T' chứa đầy đủ mọi thay đổi xảy ra trong thời gian build.

(b) **Swap** hoán đổi con trỏ gốc từ T sang T' bằng $O(1)$, giải phóng bộ nhớ cây cũ T , và xoá operation logger.

3.5 Direct Point Registration

Các hệ thống LiDAR odometry thế hệ trước (LOAM, LeGO-LOAM, LIO-SAM) đều yêu cầu bước feature extraction (trích xuất đặc trưng) trước khi đối sánh scan với bản đồ. Bước này phân loại điểm thành edge feature (đặc trưng cạnh, nơi độ cong cao) và planar feature (đặc trưng mặt phẳng, nơi bề mặt phẳng) dựa trên local smoothness (độ mượt cục bộ). Chỉ các điểm đặc trưng mới tham gia vào quá trình đối sánh, còn lại bị loại bỏ.

Cách tiếp cận này tạo ra sự phụ thuộc trực tiếp vào mẫu quét (scan pattern) và mật độ điểm của từng loại cảm biến. Một bộ trích xuất được tinh chỉnh cho spinning LiDAR, vòng quét đều và ring cố định sẽ không hoạt động tốt với solid-state LiDAR, mẫu quét không lặp lại, cho FoV nhỏ cũng như mật độ cao tại vùng trung tâm. Ngoài ra, trong structure-less

environment (môi trường ít cấu trúc), feature extraction tìm được rất ít điểm đặc trưng, làm giảm nghiêm trọng độ chính xác đối sánh.

Layer 1 sẽ loại bỏ hoàn toàn bước feature extraction này. Ứng với mỗi điểm thô (raw point) trong scan được đăng ký trực tiếp vào bản đồ bằng giải thuật kNN kết hợp point-to-plane residual (phần dư điểm-tới-mặt phẳng).

(a) Tận dụng mọi chi tiết hình học mà feature extraction bỏ sót, các bề mặt hơi cong, góc nhỏ, cấu trúc mờ đều đóng góp vào quá trình ước lượng.

(b) Hoàn toàn không phụ thuộc loại cảm biến, spinning LiDAR hay solid-state LiDAR đều được xử lý giống nhau, không cần tinh chỉnh tham số trích xuất cho từng loại.

(c) Tiết kiệm và rút ngắn thời gian tiền xử lý.

Thực nghiệm trên kiến trúc Layer 1 với ~20 chuỗi dữ liệu (dataset) từ 5 bộ dữ liệu khác nhau cho thấy phương pháp trực tiếp đạt độ chính xác tương đương hoặc cao hơn phương pháp đặc trưng trong ~90% các chuỗi. Số ít chuỗi còn lại chênh lệch không đáng kể.

3.6 Local Mapping Management

Layer 1 là hệ thống odometry (đo quãng đường) chứ không phải full SLAM, không có loop closure (phát hiện và hiệu chỉnh vòng lặp). Do đó, bản đồ không thể giữ toàn bộ lịch sử điểm vì hai lý do: bộ nhớ sẽ tăng không giới hạn, và quan trọng hơn, khi odometry drift (trôi tích lũy) lớn, các điểm cũ từ vị trí đã ghé thăm trước đó sẽ bị lệch so với vị trí thực, gây ra false match (đối sánh sai) khi robot quay lại vùng đó.

Giải pháp là duy trì bản đồ trong một vùng hình hộp cuboid có kích thước $L \times L \times L$ quanh vị trí hiện tại của UAV. Cơ chế dịch chuyển vùng bản đồ hoạt động như sau.

Hệ thống đặt một detection ball (quả cầu phát hiện) có bán kính $r = \gamma \times R$ tại vị trí UAV, trong đó R là tầm đo hiệu dụng của LiDAR và $\gamma > 1$ là hệ số dự phòng. Khi di chuyển, detection ball cũng di chuyển theo. Khi detection ball chạm biên của vùng bản đồ, vùng bản đồ dịch chuyển một khoảng $d = (\gamma - 1) \times R$ theo hướng tăng khoảng cách giữa detection ball và biên đã chạm. Sau khi dịch chuyển, các điểm nằm trong phần chênh lệch



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 23/53

giữa vùng cũ và vùng mới subtraction area sẽ bị xoá khỏi ikd-Tree qua Box-wise Delete. Nhờ độ phức tạp $O(\log n)$ của thao tác này, kích thước vùng bản đồ L có thể đặt rất lớn mà không ảnh hưởng đáng kể đến thời gian xử lý.

Garbage Collection (Cơ chế dọn rác bộ nhớ với α_{del}): thao tác Box-wise Delete trên ikd-Tree sử dụng cơ chế xóa lười (Lazy Labeling). Khi đám mây điểm bị loại khỏi vùng bản đồ, chúng không bị xóa vật lý khỏi RAM ngay lập tức mà chỉ được gắn cờ là "đã chết" (dead nodes). Việc UAV di chuyển liên tục qua không gian rộng sẽ khiến lượng dead nodes tích lũy rất lớn, làm tăng độ sâu của cây giả tạo và làm chậm tốc độ truy vấn kNN.

Để giải quyết vấn đề phân mảnh bộ nhớ, kiến trúc áp dụng delete criterion (tiêu chí xóa) với tham số α_{del} . Khi tỷ lệ dead nodes trên một nhánh cây vượt ngưỡng α_{del} , mặc định cấu hình ở mức 0.5, tức 50% nhánh là rác, hệ thống lập tức kích hoạt cờ tái thiết (rebuild flag). Luồng background thread (chạy ngầm) sẽ tiến hành Parallel Re-building cho nhánh cây đó, thu gom và giải phóng hoàn toàn vùng nhớ vật lý của các dead nodes, trả lại một cây con cân bằng hoàn hảo. Sự kết hợp giữa Box-wise Delete (giới hạn không gian) và tiêu chí α_{del} (giải phóng bộ nhớ) giúp Layer 1 hoạt động bền bỉ, không tạo ra tình trạng tràn vùng nhớ khi hoạt động xuyên suốt trong thời gian dài.



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 24/53

IV. Layer 2: VoxMap Manager

Lớp quản lý bản đồ voxel chiếm dụng (occupancy voxel map) cho hiệu năng thời gian thực xác định (deterministic real-time). Được xây dựng và tái thiết kế với nhiều tối ưu hóa cho bài toán điều hướng thời gian thực trên phần cứng nhúng.

Layer 2 nhận đám mây điểm đã đăng ký từ Layer 1 và xây dựng bản đồ chiếm dụng (occupancy map) phục vụ điều hướng. Trong khi bản đồ của Layer 1 là tập hợp điểm 3D thuần túy (chỉ biết “đây có điểm”, không biết “đây có vật cản hay trống”), Layer 2 rời rạc hóa không gian thành lưới các ô lập phương voxel và gán cho mỗi ô một giá trị xác suất: ô này có vật cản hay không? **Navigation planner** cần câu trả lời này cho việc phân biệt vùng trống, vùng có vật cản để xử lý và hoạch định đường đi một cách tốt nhất.

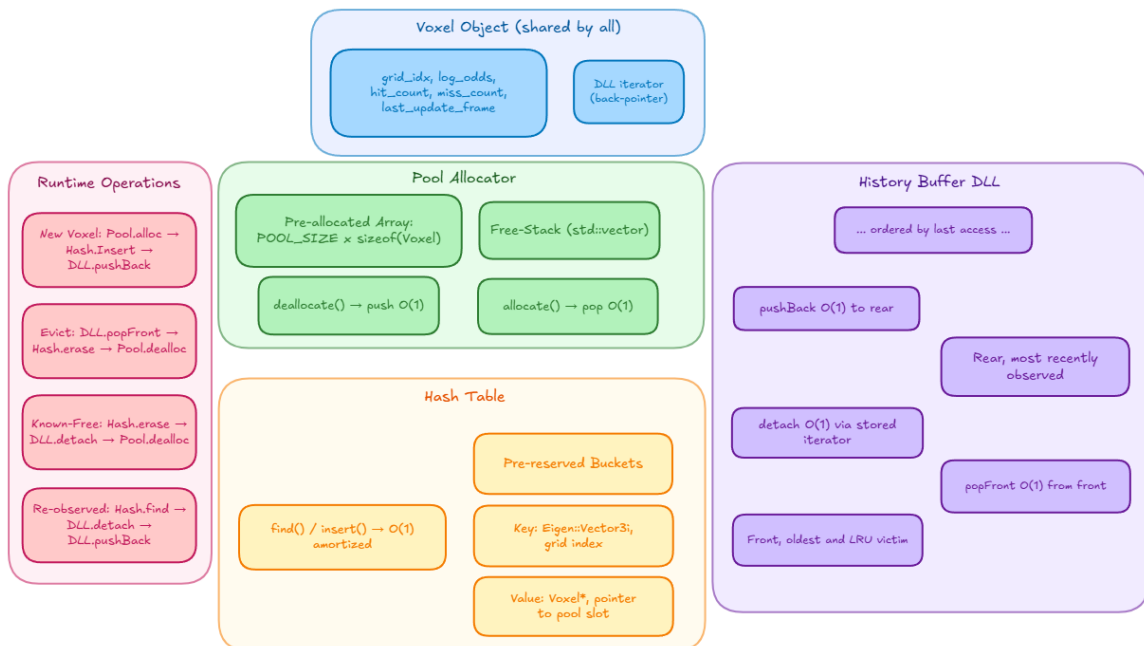
Kiến trúc Layer 2 là một hệ thống bản đồ sử dụng hash table (bảng băm) làm nền tảng lưu trữ, kết hợp quản lý voxel động theo ưu tiên không-thời gian. Kiến trúc đề xuất bốn thành phần cốt lõi chính.

(1) **Map container:** hai bảng băm riêng biệt, occupancy map lưu trạng thái chiếm dụng của voxel, mở rộng vùng cấm quanh vật cản. Cả hai chia sẻ cùng voxel object thông qua con trỏ, nên mọi cập nhật trên một bản đồ đều tự động phản ánh sang bản đồ kia mà không cần đồng bộ thủ công.

(2) **Voxel object:** mỗi voxel là một object độc lập chứa toàn bộ thông tin của chính nó, bao gồm hash key, xác suất chiếm dụng dạng log-odds, bộ đếm hit/miss, inflation look-up table, và iterator trở ngược vào history buffer. Hệ thống chỉ tạo một voxel object duy nhất cho mỗi hash key rồi phân phối bằng nhiều con trỏ đến các map container và buffer khác nhau. Nhờ thiết kế pointer-based (dựa trên con trỏ), bất kỳ cập nhật nào trên voxel đều lan truyền tức thì đến mọi nơi tham chiếu nó.

(3) **Task buffer:** ba buffer phân loại voxel trong quá trình cập nhật. B_{new} chứa voxel mới xuất hiện lần đầu trong frame hiện tại, B_{keep} chứa voxel đã tồn tại và được quan sát lại, B_{del} chứa voxel known-free (xác nhận trống) cần xoá khỏi bản đồ. Việc phân loại này cho phép mỗi nhóm voxel đi qua luồng xử lý riêng biệt, tránh kiểm tra điều kiện thừa.

(4) **History buffer:** một DLL (Doubly-Linked List, danh sách liên kết đôi) lưu lịch sử cập nhật voxel theo thứ tự thời gian. Voxel mới được chèn vào rear (cuối) danh sách, voxel được quan sát lại sẽ bị tách khỏi vị trí cũ và chèn lại vào rear, do đó voxel cũ nhất luôn dồn dần về front (đầu). Khi tổng số voxel vượt giới hạn n_{lim} , hệ thống loại voxel từ front. Mọi thao tác trên DLL đều là $O(1)$ nhờ đặc tính cấu trúc liên kết đôi. Đây là cơ chế sequential voxel removal (loại voxel tuần tự) theo ưu tiên thời gian.



Memory Architecture (Pool + Hash Table + DLL)



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 26/53

Kiến trúc Layer 2 giữ nguyên bốn thành phần cốt lõi trên nhưng được tái thiết kế cho bài toán điều hướng thời gian thực trên phần cứng nhúng với một số thay đổi quan trọng.

(a) Triển khai raycasting bằng giải thuật 3D DDA (Digital Differential Analyzer). Mỗi tia được phóng từ vị trí cảm biến đến điểm đo, đánh dấu các voxel trung gian dọc tia là free (trống) PROB_MISS và voxel tại điểm cuối là occupied (có vật cản) PROB_HIT. Cơ chế này cung cấp free-space carving chính xác về mặt vật lý: khi vật thể động di chuyển đi, tia quét từ frame tiếp theo xuyên qua voxel cũ và chủ động giảm log-odds, xoá ghost artifact trong vòng một frame duy nhất. Để kiểm soát chi phí raycasting, hệ thống giới hạn MAX_RAY_LENGTH (chiều dài tia tối đa) và chỉ cập nhật miss cho voxel đã tồn tại thay vì tạo voxel mới cho không gian trống, tránh bùng nổ bộ nhớ dọc mỗi tia.

(b) Điều chỉnh mô hình theo chiến lược High-Confience Single-Observation Model khi cho phép các voxel chuyển sang trạng thái occupied ngay từ frame đầu tiên bởi dữ liệu đầu vào đã được Layer 1 lọc nhiễu qua IEKF và point-to-plane matching.

(c) Bổ sung pool allocator (bộ cấp phát theo nhóm) để loại bỏ heap allocation khỏi runtime, đảm bảo latency (độ trễ) xác định.

(d) Sử dụng processing timeout (hết giờ xử lý) làm safety net (lưới an toàn) đảm bảo hàm update() không vượt ngân sách thời gian.

(e) Thêm frame-age eviction (loại theo tuổi frame) để dọn dẹp voxel nằm ngoài trường nhìn lâu dài mà lazy decay không chạm tới.

Luồng xử lý tổng thể của Layer 2 mỗi khi nhận dữ liệu đã đăng ký từ Layer 1 như sau:

(1) **Discretization & Hash Lookup:** mỗi điểm 3D trong không gian sẽ được chuyển thành grid index (chỉ số lưới) rồi đưa qua hàm băm để tìm voxel tương ứng trong hash table. Nếu voxel chưa tồn tại, hệ thống cấp phát từ pool allocator, khởi tạo log-odds = 0, chèn vào hash table và phân loại vào B_{new} . Nếu voxel đã tồn tại, phân loại vào B_{keep} .

(2) **Raycasting (3D DDA):** với mỗi điểm, hệ thống phóng tia từ vị trí cảm biến đến endpoint (điểm đo). Giải thuật DDA, Digital Differential Analyzer duyệt tuần tự các voxel

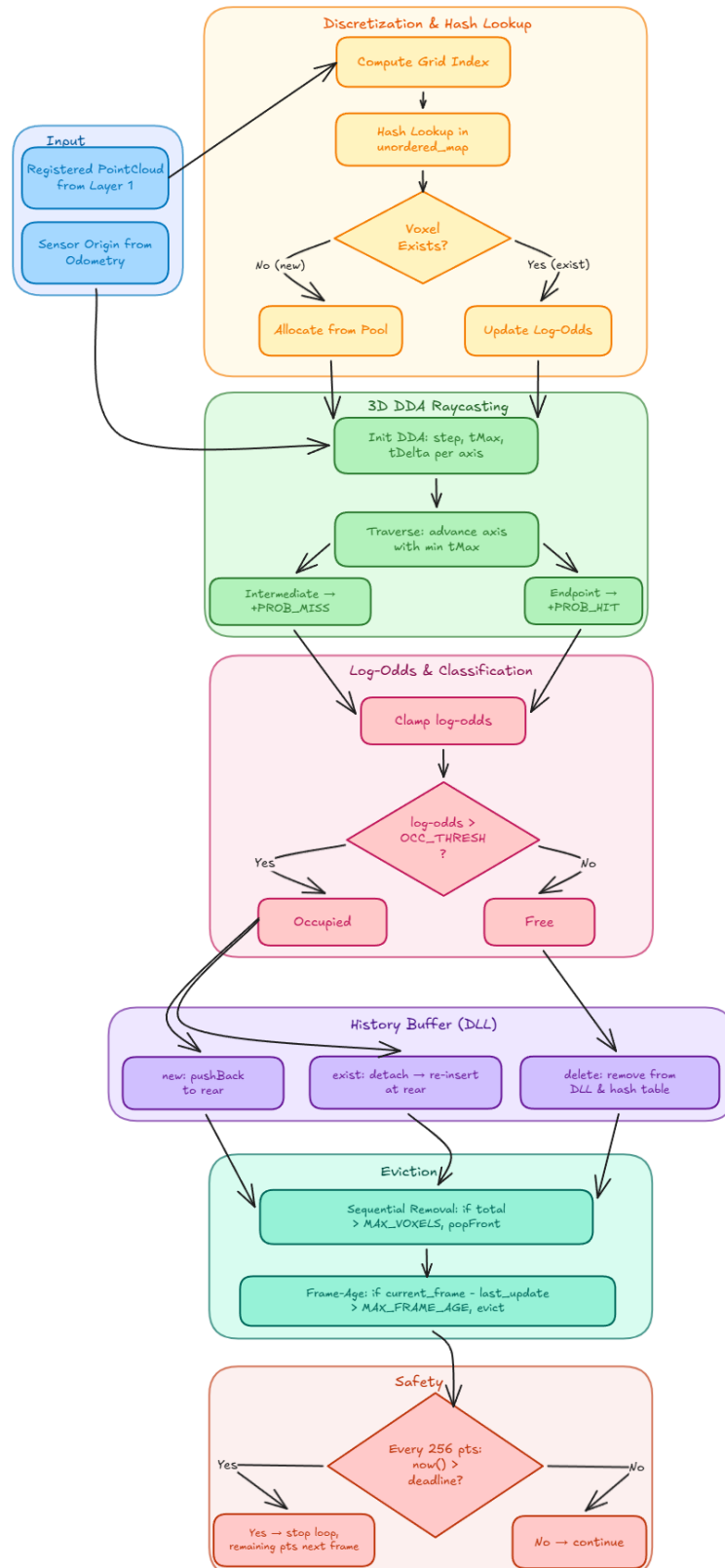
mà tia xuyên qua, cập nhật PROB_MISS cho voxel trung gian đã tồn tại (trống) và PROB_HIT cho voxel tại điểm cuối (có vật cản). Voxel trung gian chưa tồn tại trong hash table không được tạo mới để tránh bùng nổ bộ nhớ.

(3) **Log-Odds Update:** cộng PROB_HIT vào log-odds, clamp (giới hạn) giá trị trong khoảng cho phép, so sánh với OCC_THRESH (ngưỡng chiếm dụng) để xác định trạng thái occupied (có vật cản) hay free (trống), cập nhật last_update_frame.

(4) **History Buffer Update:** với voxel trong B_{new} , chèn vào rear của DLL bằng pushBack và lưu iterator vào voxel object. Với voxel trong B_{keep} , tách khỏi vị trí cũ trong DLL bằng iterator đã lưu rồi chèn lại vào rear và cập nhật iterator. Voxel known-free được chuyển vào B_{del} và xoá khỏi map container.

(5) **Sequential Voxel Removal & Eviction:** khi tổng voxel vượt n_{lim} , loại voxel từ front của DLL, xoá khỏi hash table và trả bộ nhớ về pool. Đồng thời kiểm tra frame-age, khi voxel không được quan sát quá MAX_FRAME_AGE frame cũng bị loại.

(6) **Timeout Check:** trong suốt vòng lặp bước (1)-(3), mỗi N điểm hệ thống kiểm tra thời gian. Nếu vượt deadline, vòng lặp kết thúc, điểm còn lại sẽ đến ở frame sau.



VoxMap Processing Pipeline



4.1 Spatial Hashing

Spatial hashing (bấm không gian) ánh xạ tọa độ 3D liên tục vào hash table (bảng băm) cho phép truy cập voxel với độ phức tạp $O(1)$ amortized (khấu hao). So với cấu trúc dạng tree-based như octree có truy cập $O(\log n)$, hash table cho phép truy cập và chèn nhanh hơn đáng kể khi số lượng voxel lớn. Quá trình ánh xạ gồm hai bước tuần tự.

(a) **Discretization:** sử dụng phương pháp rời rạc hóa là chia từng thành phần tọa độ cho VOXEL_RES (kích thước cạnh voxel), trừ đi gốc tọa độ bản đồ, làm tròn xuống bằng phép floor, thu được ba giá trị tọa độ gọi là grid index. Tất cả các điểm nằm trong cùng một ô lập phương sẽ có cùng grid index.

$$i_n = \text{floor}((p_n - o) / \text{res})$$

Trong đó p_n là tọa độ điểm, o là gốc tọa độ bản đồ, và res là kích thước cạnh voxel.

(b) **Hashing:** sử dụng phương pháp kết hợp tuần tự theo mô hình Boost hash_combine để ánh xạ tọa độ lưới 3D rời rạc Eigen::Vector3i sang bộ nhớ một cách nhanh chóng. Mỗi thành phần tọa độ grid (x, y, z) được băm qua std::hash rồi tích lũy vào giá trị hash chung thông qua hằng số golden ratio kết hợp bit-shift, đảm bảo các voxel lân cận trong thực tế được phân phối rải rác và đồng đều trên toàn bộ không gian bucket của bộ nhớ.

Ngăn chặn Rehash và Suy biến $O(n)$: kiến trúc nền tảng sử dụng std::unordered_map, mặc định, khi số phần tử vượt quá load factor (hệ số tải), unordered_map tự động rehash (băm lại), phải cấp phát lại toàn bộ bucket và ánh xạ lại mọi phần tử với chi phí $O(n)$. Trong một hệ thống SLAM thời gian thực, một lần rehash đột ngột chắc chắn sẽ phá vỡ "hard deadline" của quá trình xử lý Raycasting.

Để loại bỏ hoàn toàn rủi ro này và đảm bảo mọi thao tác tra cứu (lookup) và chèn (insert) đều đạt tốc độ $O(1)$ amortized, kết hợp hai chiến lược:

(a) **Khởi tạo pre-reserve (cấp phát trước không gian):** loại bỏ rủi ro rehash trong quá trình runtime.

(b) **Kiểm soát hash collision (va chạm hàm băm):** Việc cấp phát trước giúp tránh rehash, nhưng nếu xảy ra va chạm băm, nhiều phần tử rơi vào cùng một bucket tạo thành danh sách liên kết dài làm giảm tốc độ truy xuất và có thể suy biến thành $O(n)$. Bằng cách thiết lập `max_load_factor`, kết hợp với cơ chế dọn rác LRU luôn ghim số lượng voxel vật lý tối đa ở mức `MAX_VOXELS`. Khoảng sparse bucket space (không gian thưa thớt) sẽ triệt tiêu tối đa xác suất xảy ra chuỗi va chạm, giúp ổn định độ phức tạp $O(1)$ thực tế của hệ thống.

4.2 History Buffer & Sequential Voxel Removal

Hash table không có cơ chế tự quản lý vòng đời voxel. Khác với dạng array-based map (bản đồ dạng mảng) có biên không gian cố định tự động loại bỏ dữ liệu ngoài vùng, hash table chỉ lưu và truy xuất, không biết voxel nào nên giữ và voxel nào nên xóa. Cần một history buffer (một nhớ đệm) sử dụng **DLL**, **Doubly-Linked List** (danh sách liên kết đôi) lưu lịch sử cập nhật voxel theo thứ tự thời gian. Mỗi voxel object lưu một iterator trở ngược vào vị trí của chính nó trong DLL, cho phép tách và chèn lại với chi phí $O(1)$ mà không cần tìm kiếm.

Voxel mới B_{new} : nếu trạng thái là occupied, chèn vào rear của DLL bằng `pushBack`, lưu iterator vừa tạo vào voxel object. Nếu trạng thái là known-free, chuyển vào B_{del} để xóa.

Voxel đã tồn tại B_{keep} : tách khỏi vị trí hiện tại trong DLL bằng iterator (bộ lặp) đã lưu với chi phí $O(1)$, nếu vẫn còn occupied thì chèn lại vào rear và cập nhật iterator mới. Nếu chuyển sang known-free thì chuyển vào B_{del} .

DLL luôn duy trì thứ tự theo thời gian: voxel vừa quan sát nằm ở rear, voxel lâu không được quan sát dần dồn về front. Khi tổng số voxel vượt giới hạn n_{lim} , hệ thống loại voxel từ front bằng `popFront`, xóa khỏi hash table và trả bộ nhớ về pool. Quá trình loại dừng lại khi tổng voxel giảm về dưới n_{lim} . Toàn bộ cơ chế có chi phí $O(1)$ cho mỗi thao tác nhờ tính đặc trưng của **Linked Lists** (danh sách liên kết).

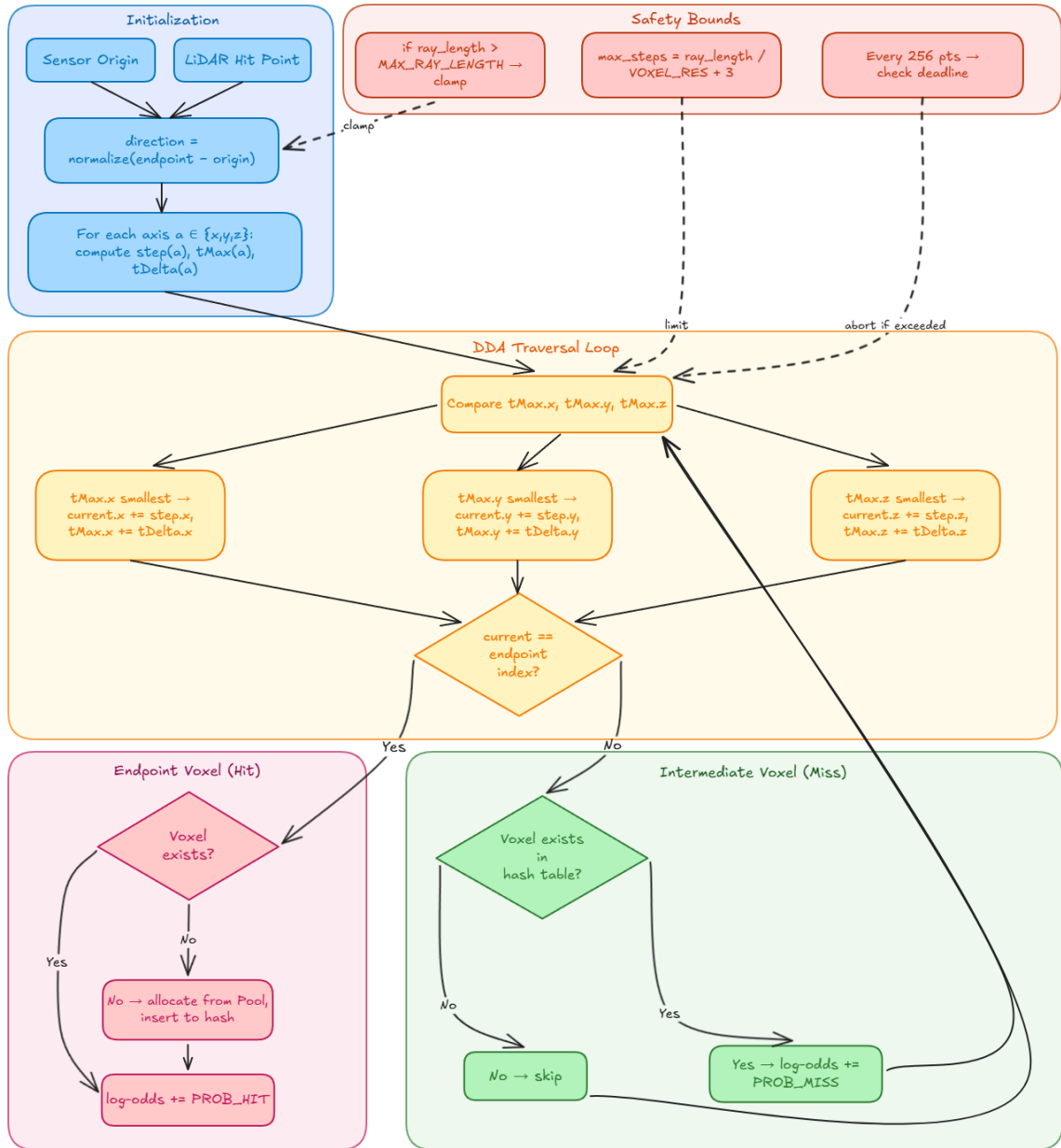
4.3 Pool Allocator

Heap allocation (cấp phát bộ nhớ động) thông qua new/malloc là nguồn gốc chính của jitter (biến động thời gian) trong hệ thống thời gian thực. Mỗi lần gọi, allocator phải tìm khối trống phù hợp với chi phí biến đổi, có thể gọi syscall (lời gọi hệ thống) với chi phí cao, và gây các lỗi page fault (hệ điều hành cấp phát trang bộ nhớ vật lý mới) với chi phí cao và không dự đoán được. Ngoài ra, cấp phát và giải phóng liên tục gây memory fragmentation (phân mảnh bộ nhớ khiến allocator phải tìm kiếm khối trống) và lock contention (tranh chấp khoá khi nhiều luồng cùng gọi malloc) trong môi trường đa luồng.

Pool allocator (bộ cấp phát theo nhóm) tách biệt hoàn toàn việc cấp phát bộ nhớ khỏi runtime. Tại thời điểm khởi tạo, hệ thống gọi malloc một lần duy nhất để cấp phát một khối bộ nhớ liên tục có kích thước $POOL_SIZE \times \text{sizeof}(\text{Voxel})$. Khối bộ nhớ này được chia thành các slot có kích thước cố định, quản lý bằng free-stack (ngăn xếp slot trống). Cấp phát voxel mới tương đương pop (lấy slot trống ở đỉnh ngăn xếp), giải phóng voxel tương đương push (đẩy slot về đỉnh ngăn xếp), cả hai đều là $O(1)$ cố định. Sau khởi tạo, toàn bộ runtime loop không có syscall nào, không có fragmentation, không có lock contention. Đây là yếu tố then chốt đảm bảo latency (độ trễ) xác định trên phần cứng nhúng.

4.4 3D DDA Raycasting

Raycasting là cơ chế cốt lõi cho phép bản đồ chiếm dụng phản ánh chính xác trạng thái thực của môi trường, đặc biệt khi có vật thể động. Nguyên lý vật lý rất trực quan: khi cảm biến LiDAR phát xung laser và nhận phản xạ tại một điểm, toàn bộ không gian từ cảm biến đến điểm phản xạ đó chắc chắn phải trống, vì xung laser đã đi qua mà không bị chặn. Đây là thông tin free-space (không gian trống) tự nhiên từ phép đo LiDAR mà hệ thống cần khai thác. Khi vật thể động di chuyển đi, tia quét từ frame (khung) tiếp theo sẽ xuyên qua vùng voxel cũ đã đánh dấu occupied (bị chiếm dụng), chủ động giảm log-odds của chúng qua PROB_MISS, xoá ghost artifact (vết ma) trên bản đồ trong vòng một hoặc vài frame tùy thuộc tốc độ tích lũy log-odds trước đó.



3D DDA Raycasting Algorithm

Kiến trúc Layer 2 triển khai raycasting dùng **3D DDA, Digital Differential Analyzer**, một giải thuật kinh điển trong đồ họa không gian cho phép duyệt chính xác tất cả voxel trong lưới không gian 3D đồng đều. Với mỗi điểm đo trong scan, hệ thống phóng một tia từ vị trí cảm biến, lấy từ pose đã ước lượng bởi Layer 1 đến endpoint, sau đó sử dụng DDA duyệt tuần tự từng voxel dọc theo tia, gồm ba giai đoạn chính.

(a) **Khởi tạo DDA:** hệ thống tính vector hướng (direction) từ origin đến endpoint, chuẩn hoá (normalize) về độ dài đơn vị, và xác định ba bộ tham số cho ba trục x, y, z. Hướng bước trên trục a (+1 hoặc -1 tùy hướng tia). $tMax(a)$ là khoảng cách dọc tia đến biên voxel đầu tiên trên trục a. $tDelta(a)$ là khoảng cách dọc tia để đi hết một voxel trên trục a, bằng $VOXEL_RES / |direction(a)|$. Nếu thành phần hướng trên trục a đạt mức xấp xỉ 0 (tia song song trục đó), $tMax$ và $tDelta$ được đặt bằng vô cực để trục đó không bao giờ được chọn trong bước duyệt.

(b) **Duyệt DDA (DDA traversal):** bắt đầu từ voxel chứa sensor origin, giải thuật lặp và tại mỗi bước chọn trục có $tMax$ nhỏ nhất để bước tiến (advance). Nếu $tMax.x < tMax.y$ và $tMax.x < tMax.z$ thì bước theo trục x, cập nhật $current.x += step.x$ và $tMax.x += tDelta.x$, tương tự cho trục y và z. Mỗi voxel trung gian mà tia laser xuyên qua đại diện cho không gian trống, cộng $PROB_MISS$ vào log-odds. Khi current index trùng với endpoint index, tăng $PROB_HIT$ và dừng. Safety bound (giới hạn an toàn) với $max_steps = ray_length / VOXEL_RES + 3$ được sử dụng để giới hạn tổng số bước duyệt nhằm tránh vòng lặp vô hạn trong trường hợp biên số học.

(c) **Cập nhật voxel:** quá trình cập nhật tại điểm cuối tia và tại voxel trung gian tuân theo hai quy tắc khác nhau. Tại điểm cuối tia hit, nếu voxel chưa tồn tại trong hash table, hệ thống cấp phát voxel mới từ pool, chèn vào hash table và phân loại vào B_{new} , và nếu voxel đã tồn tại thì cập nhật log-odds và phân loại vào B_{keep} . Tại voxel trung gian miss, hệ thống không tạo voxel mới, nếu voxel không tồn tại trong hash table thì bỏ qua hoàn toàn, chỉ cộng $PROB_MISS$ khi voxel đã có sẵn. Nhờ vậy khi hệ thống chỉ xoá vật chiếm dụng cũ tại đúng nơi cần thiết mà không tiêu tốn bộ nhớ cho không gian vốn đã trống.

4.5 Log-Odds Occupancy Model

Mỗi voxel lưu một giá trị log-odds (tỉ lệ log) biểu diễn xác suất chiếm dụng dưới dạng logarithm, theo mô hình xác suất *Bayesian*.

$$l = \log(p / (1 - p))$$



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 34/53

Trong đó p là xác suất voxel chứa vật cản. Giá trị $l = 0$ tương ứng với trạng thái chưa biết ($p = 0.5$), $l > 0$ nghiêng về occupied, $l < 0$ nghiêng về free.

Ưu điểm cốt lõi của log-odds so với xác suất trực tiếp là phép cập nhật Bayesian trở thành phép cộng đơn thuần thay vì phép nhân và chuẩn hoá, vừa nhanh vừa ổn định số học không gặp underflow (tràn dưới) khi xác suất rất nhỏ.

Kiến trúc Layer 2 cập nhật log-odds bằng cả hit count (đếm trúng, khi điểm rơi vào voxel) và miss count (đếm trượt, khi tia đi qua voxel) thông qua Raycasting.

$$L(n | s_t) = n_{\text{hit}} \times l_{\text{hit}} + n_{\text{miss}} \times l_{\text{miss}}$$

$$L(n | s_{1:t}) = L(n | s_{1:t-1}) + L(n | s_t)$$

Cơ chế cập nhật qua 3D DDA Raycasting:

Mô hình Bayesian được triển khai thông qua thuật toán 3D DDA. Khi một tia laser được nội suy từ tâm cảm biến đến điểm phản xạ. Voxel tại điểm cuối Endpoint nhận PROB_HIT (có vật cản), còn các voxel trung gian mà tia xuyên qua nhận PROB_MISS (trống). Một voxel được coi là occupied khi log-odds vượt OCC_THRESH (ngưỡng chiếm dụng).

Chiến lược High-Confidence Single-Observation:

Thay vì áp dụng tích lũy đa quan sát **Multi-hit** đồng đều, hệ thống sử dụng chiến lược tích lũy bất đối xứng, tận dụng tối đa dữ liệu đã được tinh lọc kỹ lưỡng từ bộ lọc IEKF và point-to-plane matching của Layer 1.

(a) **Fast Confirmation:** cho phép bản đồ phản ứng tức thời với cả sự xuất hiện lẫn sự biến mất của vật cản, vật thể mới được xác nhận ngay khi tia phản xạ.

(b) **Conservative clearing:** đối với các khu vực đã được xác nhận là trống trải nhiều lần, log-odds bị ép xuống mức tối thiểu. Nếu xuất hiện nhiễu hoặc điểm ảo, hệ thống đòi hỏi phải có nhiều lần tia hit liên tiếp mới đủ sức lật ngược trạng thái thành Occupied nhưng vẫn đáp ứng đủ nhanh cho việc điều hướng nếu có vật thể động di chuyển vào. Ngược lại,

voxel đã bị chiếm dụng lâu cũng cần nhiều tia miss liên tiếp để xác nhận lại trạng thái. Điều này mang lại sự bảo vệ vững chắc trước các lỗi đo lường cục bộ.

Bên cạnh đó, để giải quyết bài toán Dynamic Objects (vật thể động), giá trị log-odds sẽ được clamp (giới hạn) trong khoảng [CLAMP_MIN, CLAMP_MAX] để tránh tình trạng over-confident (quá tự tin). Nếu không giới hạn, voxel đã được quan sát nhiều lần sẽ tích lũy log-odds rất cao, cần rất nhiều bằng chứng ngược lại mới thay đổi trạng thái, khiến hệ thống phản ứng chậm khi môi trường thay đổi. Cơ chế Clamping đảm bảo hệ thống duy trì khả năng "quên" đủ nhanh, giúp bản đồ linh hoạt thích ứng với các thay đổi liên tục của môi trường thực tế.

4.6 Processing Timeout

Dù mỗi thao tác riêng lẻ là $O(1)$, tổng thời gian xử lý vẫn tỉ lệ tuyến tính với số điểm đầu vào $O(n)$. Trong điều kiện bất thường (nhiều multipath, môi trường dày đặc), scan có thể chứa gấp đôi số điểm trung bình. Processing timeout (hết giờ xử lý) đóng vai trò safety net (lưới an toàn) đảm bảo hàm update() không bao giờ vượt quá UPDATE_TIMEOUT_US.

Trước vòng lặp, hệ thống ghi deadline = now() + UPDATE_TIMEOUT_US. Trong vòng lặp, với mỗi N điểm, hệ thống so sánh thời gian hiện tại với deadline, nếu vượt quá, kết thúc ngay lập tức. Các điểm đã xử lý vẫn đúng và được cập nhật vào bản đồ, các điểm bị bỏ qua sẽ đến lại ở frame sau.

4.7 Frame-Age Eviction

Frame-age eviction (loại bỏ theo thời gian tồn tại của frame) bổ sung cho hai cơ chế trước đó. Sequential voxel removal chỉ kích hoạt khi tổng voxel vượt n_{lim} , raycasting chỉ xóa bằng chứng chiếm dụng khi tia xuyên qua voxel đã tồn tại. Voxel nằm ngoài **FOV**, **Field of View** (trường nhìn) lâu dài sẽ không bị raycasting chạm tới và chưa chắc bị loại nếu bản đồ chưa đầy. Frame-age eviction loại vô điều kiện mọi voxel không được quan sát quá MAX_FRAME_AGE frame.

Khi DLL đã sắp xếp theo thứ tự truy cập, front luôn là voxel cũ nhất, hệ thống chỉ cần kiểm tra tuần tự từ front: nếu $current_frame - last_update_frame > MAX_FRAME_AGE$ thì

loại voxel, xoá khỏi hash table và trả bộ nhớ về pool, rồi tiếp tục kiểm tra node tiếp theo. Khi gặp voxel chưa vượt tuổi thì dừng, vì mọi node phía sau nó trong DLL đều mới hơn. Chi phí là $O(k)$ với k là số voxel bị loại, thường không lớn trong mỗi khung frame.

4.8 Incremental Map Broadcast (Optional)

Đề xuất tính năng map-sharing sử dụng circular buffer (bộ đệm vòng) để lưu và truyền voxel mới quan sát cho các agent khác trong hệ thống đa tác nhân. Kiến trúc Layer 2 tái thiết kế tính năng này cho hai kiến trúc triển khai khác nhau.

(1) **Kiến trúc đơn tiến trình (single-process):** planner truy vấn trực tiếp hash table của Layer 2 qua shared memory (bộ nhớ chia sẻ) hoặc API call trong cùng process, không cần publish bản đồ và chi phí truyền bằng 0.

(2) **Kiến trúc đa tiến trình (multi-process):** planner chạy node ROS riêng hoặc khi cần broadcast bản đồ cho nhiều module, việc truyền bản đồ qua ROS middleware là cần thiết.

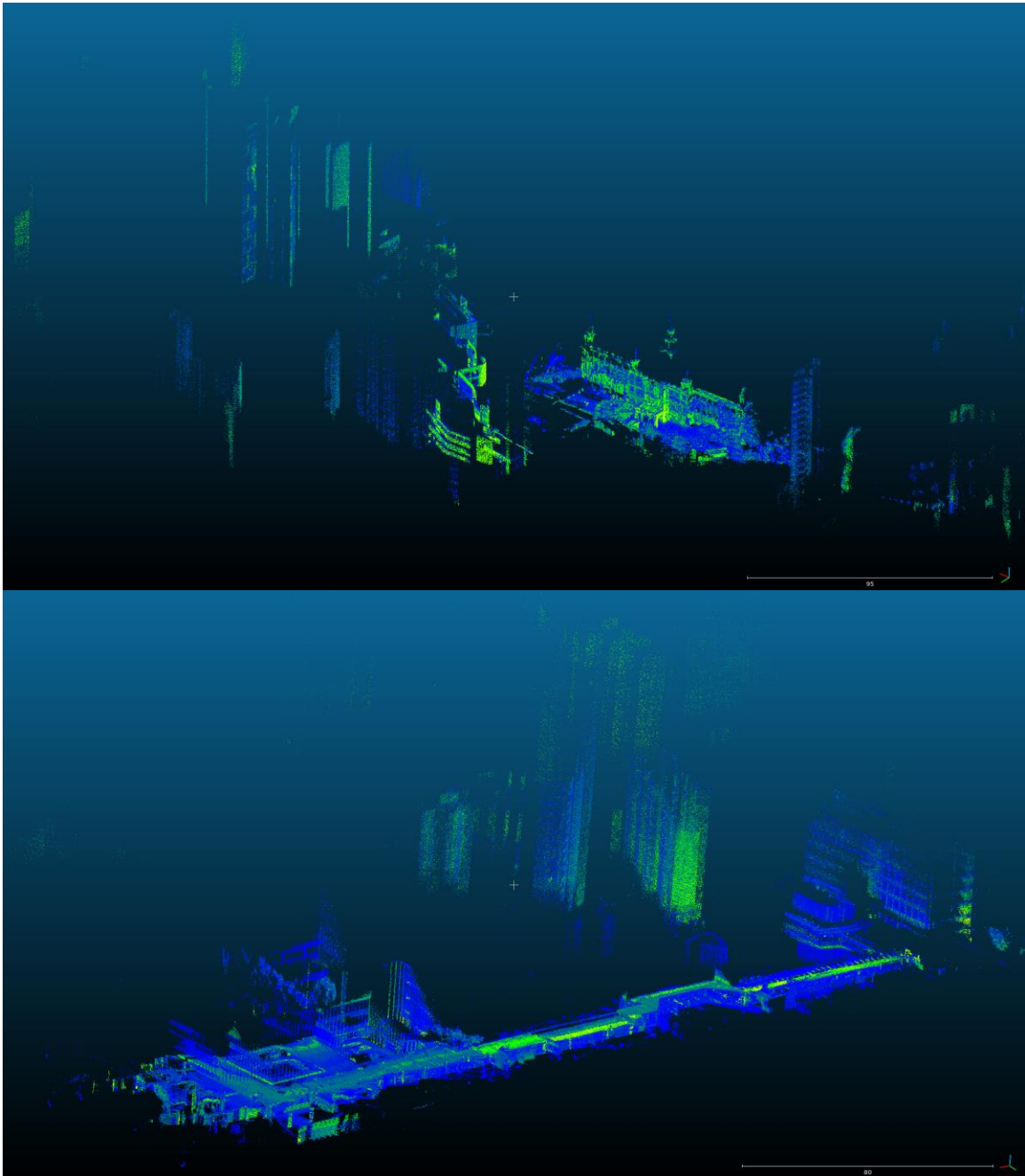
Phương pháp naïve là duyệt toàn bộ bản đồ và gửi tất cả voxel mỗi frame, chi phí tỉ lệ tổng số voxel trong bản đồ. Giải pháp incremental publish (phát tăng dần), hệ thống tận dụng hai task buffer B_{new} và B_{keep} đã có sẵn từ quá trình cập nhật, chỉ publish voxel trong hai buffer này thay vì toàn bộ bản đồ. Phía nhận tích lũy các incremental update để dựng lại bản đồ đầy đủ. Tối ưu này chỉ ảnh hưởng đến chi phí truyền bản đồ qua middleware, không ảnh hưởng đến chi phí update() cốt lõi. Trong kiến trúc đơn tiến trình với truy vấn trực tiếp, tính năng này có thể tắt hoàn toàn.

V. Complexity Summary

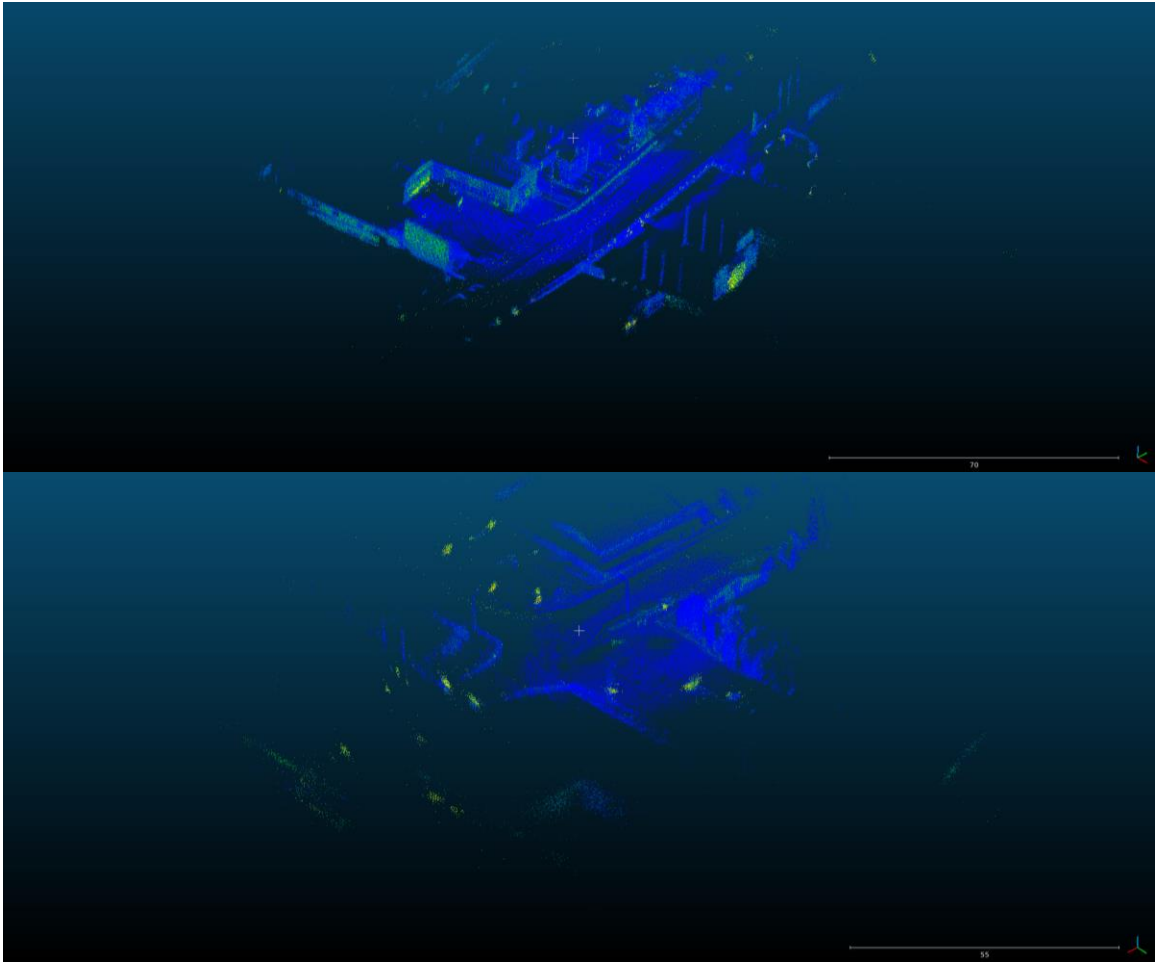
Thao tác	Time Complexity	Ghi chú
Hash lookup/insert	$O(1)$ amortized	Pre-reserved buckets
DLL attach/detach	$O(1)$	Pointer swap
Pool allocate/deallocate	$O(1)$	Free-stack pop/push
Raycasting (3D DDA)	$O(n \times d_{avg})$	Hit/Miss voxel
LRU eviction	$O(1)$ per voxel	Remove front
Frame-age eviction	$O(\text{evicted})$	Traverse DLL from front
Timeout check	$O(1)$ per 256 pts	Bitwise AND
Full update	$O(n)$ $n = \text{input pts}$	Full functions
Map broadcast	$O(\text{changed})$	Publish $B_{\text{keep}} + B_{\text{new}}$
IEKF	$O(m \cdot \log(N) + K \cdot \text{dim}^2 \cdot m)$	5-NN search + 24×24 Kalman gain
ikd-Tree insert	$O(\log n)$	On-tree downsampling
ikd-Tree kNN	$O(\log n)$	Bounds-overlap-ball pruning
ikd-Tree box-delete	$O(\log n)$	Lazy labels + Bounding box pruning

VI. Benchmark Results

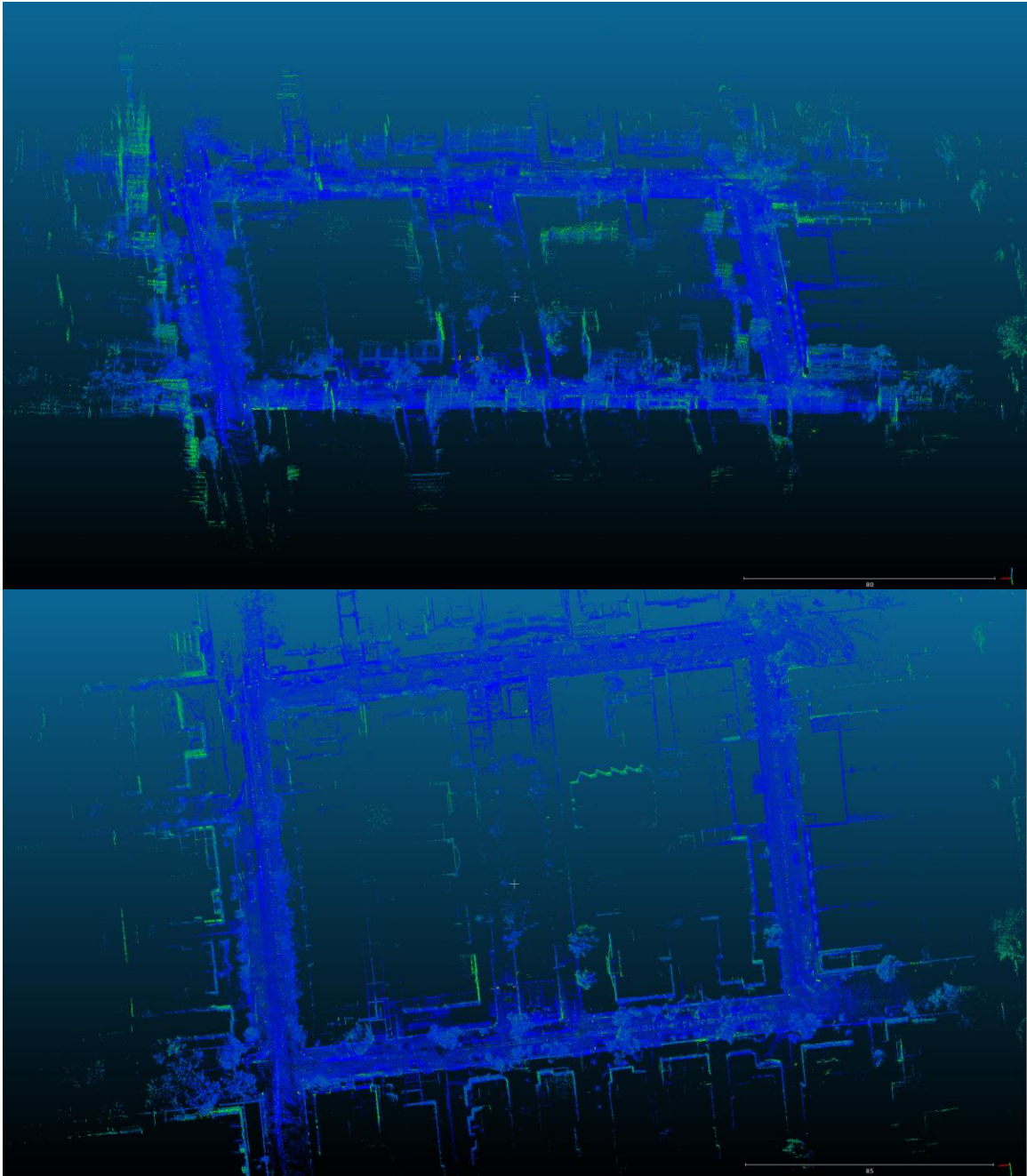
Benchmark được trong 20 lần kiểm thử, sử dụng rosbag replay để mô phỏng kiểm thử hệ thống với giao thức Intra-process zero-copy IPC giữa 2 Layer, độ trễ $\sim 0ms$.



Testcase 1-2: Livox AVIA LiDAR



Testcase 3: Livox Mid360 LiDAR



Testcase 4: Robosense RS-LiDAR-32

6.1 Layer 1 Performance

Metric (mean/run)	Case 1	Case 2	Case 3	Case 4
Type	AVIA	AVIA	Mid360	RS-LiDAR-32
Environment	University 1	University 2	Traffic 1	Traffic 2
Frequency	LiDAR 10Hz IMU 75Hz	LiDAR 100Hz IMU 200Hz	LiDAR 10Hz IMU 200Hz	LiDAR 10Hz IMU 100Hz
Total Frames	~1408.2	~35052.4	~652.6	~2517.0
Total Time	~7288.733 ms	~33856.907 ms	~2418.924 ms	~32129.704 ms
Min Process Time	~1.940 ms	~0.148 ms	~2.434 ms	~6.891 ms
Max Process Time	~21.800 ms	~12.130 ms	~8.923 ms	~130.766 ms
Mean Process Time	~5.176 ms	~0.966 ms	~3.707 ms	~12.765 ms
Points/Frame	~1560.1	~199.9	~1106.8	~3217.3
Frame > 10ms	~5.6 (0.40%)	~0.2 (0.00%)	~0.0 (0.00%)	~2378.2 (94.47%)
Frame > 20ms	~0.2 (0.01%)	~0.0 (0.00%)	~0.0 (0.00%)	~8.6 (0.34%)
Frame < 50ms	100.00%	100.00%	100.00%	~2516.0 (99.96%)



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 42/53

6.2 Layer 2 Performance

Metric (mean/run)	Case 1	Case 2	Case 3	Case 4
Type	AVIA	AVIA	Mid360	RS-LiDAR-32
Environment	University 1	University 2	Traffic 1	Traffic 2
Frequency	LiDAR 10Hz IMU 75Hz	LiDAR 100Hz IMU 200Hz	LiDAR 10Hz IMU 200Hz	LiDAR 10Hz IMU 100Hz
Total Frames	~1408.2	~35052.6	~652.6	~2516.6
Mean Update Time	~6.927 ms	~0.675 ms	~2.099 ms	~7.219 ms
Mean Publish Time	~0.211 ms	~0.068 ms	~0.109 ms	~0.281 ms
Min Total Time	~1.876 ms	~0.036 ms	~1.315 ms	~2.513 ms
Max Total Time	~23.441 ms	~5.752 ms	~8.410 ms	~70.673 ms
Mean Total Time	~7.138 ms	~0.743 ms	~2.208 ms	~7.500 ms
Points/Frame	~5857.7	~536.5	~1106.8	~3217.3
Max voxels reached	410519	868419	217257	1401566
Timeouts triggered	0.0	0.0	0.0	0.4
Frame > 5ms update	~293.4 (20.84%)	~0.0 (0.00%)	~0.0 (0.00%)	~473.2 (18.81%)
Frame > 10ms update	~1.2 (0.08%)	~0.0 (0.00%)	~0.0 (0.00%)	~13.2 (0.53%)
Frame < 50ms update	100.00%	100.00%	100.00%	~2516.2 (99.98%)



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 43/53

6.3 Middleware bandwidth & Incremental Broadcast

Metric (mean/run)	Case 1	Case 2	Case 3	Case 4
Type	AVIA	AVIA	Mid360	RS-LiDAR-32
Environment	University 1	University 2	Traffic 1	Traffic 2
Frequency	LiDAR 10Hz IMU 75Hz	LiDAR 100Hz IMU 200Hz	LiDAR 10Hz IMU 200Hz	LiDAR 10Hz IMU 100Hz
Layer 2 Publish /livox_map				
Total Size	~138.9 MB	~383.1MB	~23.2 MB	~258.6 MB
Average Msg Size	~103378 bytes	~11453 bytes	~37284 bytes	~107658 bytes
Average Bandwidth	~8.28 Mbps	~9.16 Mbps	~2.99 Mbps	~6.84 Mbps
Duration	~140.762 s	~350.771 s	~65.166 s	~316.764 s
Layer 1 Publish /livox_processed				
Total Size	~377.8 MB	~863.6 MB	~33.1 MB	~371.0 MB
Average Msg Size	~281240 bytes	~25816 bytes	~53190 bytes	~154504 bytes
Average Bandwidth	~22.51 Mbps	~20.65 Mbps	~4.26 Mbps	~9.83 Mbps
Duration	~140.755 s	~350.771 s	~65.164 s	~316.759 s

VII. Architecture Strengths

Kiến trúc 2 lớp được thiết kế với mục tiêu phục vụ obstacle avoidance và path planning cho UAV, không phải xây bản đồ toàn cục. Mọi quyết định thiết kế đều hướng tới một yêu cầu duy nhất: planner luôn nhận được bản đồ vật cản cục bộ chính xác, cập nhật, với latency xác định trong mọi điều kiện hoạt động.

7.1 Deterministic Real-Time

Hệ thống đảm bảo worst-case bounded thông qua ba tầng bảo vệ hoạt động đồng thời.

(a) **Tầng thứ nhất là processing timeout:** mỗi lần gọi update(), hệ thống đặt hard deadline bằng UPDATE_TIMEOUT_US. Nếu vòng lặp raycasting vượt thời gian cho phép, hệ thống kết thúc ngay lập tức và các điểm còn lại sẽ được xử lý ở frame tiếp theo.

(b) **Tầng thứ hai là pool allocator:** toàn bộ bộ nhớ cho voxel được cấp phát một lần tại khởi tạo, loại bỏ hoàn toàn jitter từ heap allocation vốn có thể gây spike mỗi lần gọi malloc.

(c) **Tầng thứ ba là hash table pre-reserved:** bucket được cấp phát trước, loại bỏ rủi ro rehash, một thao tác $O(n)$ có thể phá vỡ deadline nếu xảy ra giữa chừng.

Deterministic timing là yêu cầu bắt buộc cho obstacle avoidance: nếu một frame mapping trễ quá lượng thời gian được định, planner phải dùng bản đồ cũ hoặc bị starve hoàn toàn, dẫn đến quyết định điều hướng dựa trên thông tin lỗi thời. Trong môi trường có vật cản động, chỉ cần 1–2 frame trễ là UAV có thể không phát hiện vật cản mới xuất hiện. Các hệ thống mapping toàn cục như OctoMap, Voxblox, OpenVDB được thiết kế để xây bản đồ hoàn chỉnh chứ không ưu tiên worst-case latency, do đó không có cơ chế timeout hay bounded memory, worst-case latency có thể lên đến 50–200ms tùy kích thước bản đồ, đủ để gây va chạm ở tốc độ di chuyển thông thường của UAV.

Sự khác biệt ở đây là kiến trúc **VoxMap** của bài báo cáo giải quyết bài toán **local obstacle avoidance** thay vì hoàn toàn là cho mục tiêu **global reconstruction** một cách nhanh chóng. Nhưng nếu cần vẫn có thể dùng bản đồ toàn cục sơ bộ phác thảo bức tranh tổng thể về môi trường đã đi qua.

7.2 Pre-allocated Memory Model

Pool allocator cấp phát toàn bộ bộ nhớ một lần duy nhất tại thời điểm khởi tạo: một mảng liên tục $POOL_SIZE \times \text{sizeof}(\text{Voxel})$ slot, quản lý bằng free-stack. Khi cần voxel mới, pop từ đỉnh stack ($O(1)$) và khi thu hồi, push trả lại ($O(1)$). Không có lệnh new/delete nào trong toàn bộ runtime loop, loại bỏ ba nguồn gốc chính của jitter trong hệ thống thời gian thực là memory fragmentation, allocator lock contention, và page fault.

So sánh với các hệ thống mapping khác: OctoMap gọi new() cho mỗi octree node mới, mỗi frame có thể tạo ra hàng trăm đến hàng nghìn node với $O(n)$ heap allocation mỗi frame. Voxblox sử dụng block allocation nhưng vẫn gọi malloc khi mở rộng bản đồ. OpenVDB có memory pool nội bộ nhưng không bounded. Kiến trúc Layer 2 là hệ thống trong nhóm so sánh đạt được zero heap allocation trong runtime.

7.3 Multi-Sensor Platform

Các hệ thống LiDAR odometry thế hệ trước (LOAM, LeGO-LOAM, LIO-SAM) yêu cầu bước feature extraction phân loại điểm thành edge và planar feature dựa trên độ cong cục bộ. Phụ thuộc trực tiếp vào scan pattern và mật độ điểm của từng loại cảm biến.

Kiến trúc Layer 1 loại bỏ hoàn toàn bước feature extraction. Mỗi điểm thô trong scan được đăng ký trực tiếp vào bản đồ bằng kNN + point-to-plane residual trên ikd-Tree, chỉ cần tọa độ 3D và timestamp. Module preprocess đọc format raw và chuẩn hoá thành PointCloud2 thống nhất. Đã được kiểm chứng trên nhiều loại sensor như spinning và solid-state LiDAR. Khi thay đổi LiDAR, chỉ cần cấu hình tham số mà không cần sửa kiến trúc hay thuật toán.

7.4 Bounded-Memory Footprint

Cả hai Layer đều có bộ nhớ bounded, nhưng bằng hai cơ chế khác nhau.

Layer 2 sử dụng hard cap: pool allocator cố định $POOL_SIZE \times \text{sizeof}(\text{Voxel})$ slot tại khởi tạo, hash table pre-reserved với số bucket tương ứng. LRU và frame-age eviction đảm bảo tổng voxel luôn $\leq MAX_VOXELS$ bất kể thời gian hoạt động hay kích thước môi trường. Đây là bounded tuyệt đối giúp hệ thống không thể vượt quá bộ nhớ đã cấp phát.

Layer 1 sử dụng local map bounded by design: ikd-Tree duy trì bản đồ trong một vùng hình hộp cuboid kích thước $L \times L \times L$ quanh vị trí hiện tại. Khi UAV di chuyển, Box-wise Delete xóa các điểm nằm ngoài vùng mới, on-tree downsampling với `filter_size_map` đảm bảo mỗi kích thước voxel nhất định chỉ giữ đúng 1 điểm dữ liệu. Qua đó giới hạn lại số điểm tối đa trong ikd-Tree, $N_{\max} = (L / \text{filter_size_map})^3$, phụ thuộc vào cấu hình chứ không tăng theo thời gian. Cả hai lớp đều không cần giám sát bộ nhớ runtime VmRSS vì upper bound đã được xác định tại thời điểm cấu hình hệ thống.

7.5 Unidirectional Data Flow

Luồng dữ liệu giữa hai lớp là một chiều: Layer 1 $\rightarrow$ Layer 2, không có feedback loop. Layer 1 publish registered point cloud lên topic `/livox_processed`, Layer 2 subscribe và xử lý độc lập. Thiết kế này mang lại hai lợi ích cho hệ thống production.

(a) **Thứ nhất, khả năng tùy biến độc lập:** dễ dàng thay thế kiến trúc hệ thống Layer này mà không cần phải sửa lại kiến trúc hệ thống Layer kia.

(b) **Thứ hai, khả năng khoan vùng lỗi:** nếu Layer 1 diverge trong môi trường suy biến, Layer 2 chỉ nhận point cloud sai chứ không bị crash hay corrupt state nội bộ qua đó khoan vùng phần lỗi nhanh chóng và chính xác.

7.6 Scalability for Navigation Stack

Kiến trúc pipeline 2 lớp tạo ra các insertion point tự nhiên cho các module trung gian khi hệ thống mở rộng. Giữa Layer 1 và Layer 2 có thể chèn module semantic segmentation để phân loại điểm (tòa nhà, phương tiện, con người, ...) trước khi cập nhật bản đồ, cho phép Layer 2 xử lý vật cản động và tĩnh khác nhau. Sau Layer 2 có thể thêm inflation layer mở rộng vùng cấm quanh vật cản theo bán kính UAV, hoặc module chuyển đổi 3D voxel map thành 2D costmap cho planner 2D. Mỗi module bổ sung giao tiếp qua ROS topic chuẩn mà không cần sửa đổi hai lớp cốt lõi.

VIII. Limitations & Future Improvements

Mọi hệ thống kỹ thuật đều có giới hạn thiết kế. Kiến trúc 2 lớp được tối ưu cho bài toán obstacle avoidance cục bộ, không phải global SLAM. Nhiều mục dưới đây không phải hạn chế trong ngữ cảnh né vật cản mà là design tradeoff có chủ đích.

8.1 No Loop Closure (Design Tradeoff)

Layer 1 là hệ thống odometry (đo quãng đường tương đối), không phải full SLAM có loop closure. Mỗi frame, pose được ước lượng dựa trên frame trước cộng với phép đo IMU và LiDAR hiện tại. Sai số nhỏ ở mỗi frame tích lũy theo thời gian (drift), đặc biệt theo phương z (độ cao) và yaw (hướng quay). Khi UAV quay lại vị trí đã đi qua, bản đồ sẽ bị lệch so với lần đầu, hai phiên bản của cùng một bức tường có thể xuất hiện cách nhau vài centimet đến vài chục centimet tùy quãng đường đã di chuyển.

Đây là lựa chọn thiết kế có chủ đích khi bài toán **Obstacle Avoidance** chỉ cần bản đồ cục bộ nhất quán quanh UAV để xử lý navigation cục bộ (trong bán kính ~5-20m), không cần bản đồ toàn cục chính xác tuyệt đối.

8.2 No Degeneracy Detection (Layer 1)

IEKF có thể diverge (phân kỳ) trong môi trường geometric degeneracy (suy biến hình học), nơi cấu trúc hình học không cung cấp đủ ràng buộc cho tất cả 6 bậc tự do. Các tình huống điển hình như hành lang dài và hẹp, cánh đồng trống hoặc bay trên mái nhà, và đường hầm tròn. Khi xảy ra, ma trận $H^T R^{-1} H$ có eigenvalue rất nhỏ theo hướng suy biến, Kalman gain bùng nổ theo hướng đó, pose nhảy đột ngột và bản đồ bị méo.

Giải pháp: sau mỗi vòng lặp IEKF, thực hiện eigenvalue decomposition trên ma trận thông tin $H^T R^{-1} H$. Nếu eigenvalue nhỏ nhất $\lambda_{\min}$ nhỏ hơn ngưỡng cho trước, giảm Kalman gain theo eigenvector tương ứng, buộc bộ lọc tin tưởng IMU hơn theo hướng suy biến. Phương pháp này đã được chứng minh hiệu quả trong các hệ thống như LIO-SAM và FAST-LIO2 mở rộng.

8.3 Online Extrinsic Estimation (Layer 1)

Với các sensor tích hợp sẵn IMU trong thân LiDAR, extrinsic tham số ngoại đã được cố định và cấu hình cứng đầy đủ trong YAML với initial covariance cực nhỏ, IEKF tin tưởng gần tuyệt đối vào giá trị ban đầu. Trong một số trường hợp khi sử dụng LiDAR và IMU riêng biệt, extrinsic có thể bị dịch chuyển do rung lắc cơ khí hoặc đo đạc không chính xác.

Giải pháp: dùng online extrinsic estimation bằng cách tăng initial covariance cho phép IEKF tự tinh chỉnh extrinsic trong quá trình hoạt động (nếu cần).

8.4 Single-Resolution Voxel Grid (Layer 2)

VOXEL_RES cố định cho toàn bộ bản đồ và trong thực tế, vùng lân cận UAV trong khoảng cách $\sim 5\text{m}$ cần resolution cao hơn để phát hiện vật cản nhỏ như dây cáp, chân bàn, mép bậc thang, những vật thể có kích thước tương đương hoặc nhỏ hơn 0.1m sẽ bị mất trong lưới hiện tại. Ngược lại, vùng xa $\sim 10\text{-}20\text{m}$ cần resolution thấp hơn để nhận diện tường và vật cản lớn, giúp tiết kiệm bộ nhớ.

Giải pháp: triển khai multi-resolution voxel grid với 2-3 mức resolution đồng tâm quanh UAV, hoặc chuyển sang cấu trúc hierarchical như OctoMap. Tuy nhiên, multi-resolution làm tăng complexity của hash lookup và DLL management, cần đánh giá cân bằng giữa lợi ích giữa resolution và chi phí runtime.

8.5 No Semantic Information (Layer 2)

Bản đồ hiện tại chỉ lưu trạng thái occupied/free, không phân biệt loại vật cản. UAV đối xử các “vật cản” giống nhau, cả hai đều là occupied voxel. Điều này dẫn đến path planning suboptimal, robot có thể chờ đợi người đi bộ đi qua thay vì vòng tránh sớm khi “không biết đó là vật cản động sẽ tự di chuyển”, hoặc “lập kế hoạch đường đi qua vùng có vật cản tạm thời” thay vì vùng trống lâu dài.

Giải pháp: chèn module semantic segmentation, ví dụ Cylinder3D, SalsaNext, hoặc mô hình lightweight chạy trên GPU nhúng, giữa Layer 1 và Layer 2 để gán nhãn cho mỗi điểm trước khi cập nhật bản đồ. Layer 2 có thể mở rộng voxel object để lưu thêm trường semantic class, cho phép planner áp dụng chiến lược khác nhau cho từng loại vật cản.



8.6 Ghost Voxel Outside FoV (Layer 2)

Voxel nằm ngoài trường nhìn (FoV) hiện tại chỉ bị xoá sau $\text{MAX_FRAME_AGE} = 600$ frame. Trong khoảng thời gian đó, nếu vật cản đã di chuyển (người đi qua, cửa mở ra), voxel cũ vẫn đánh dấu occupied, tạo ra ghost obstacle (vật cản ma) khiến planner tránh vùng thực tế đã trống. Raycasting 3D DDA chỉ xoá voxel nằm trên đường tia trong FoV hiện tại, không ảnh hưởng đến vùng ngoài FoV.

Giải pháp: giảm MAX_FRAME_AGE để dọn dẹp nhanh hơn, nhưng cần cân bằng khi giá trị quá nhỏ sẽ xoá cả vật cản tĩnh hợp lệ (tường phía sau UAV) mà chưa được quan sát lại. Phương án tốt hơn là kết hợp thông tin từ FoV model của LiDAR nếu một voxel nằm trong FoV lý thuyết nhưng không nhận được hit hay miss nào, có thể đánh dấu là “ngờ” và giảm log-odds dần, thay vì chờ hết frame-age mới xoá.

IX. Parameter Configuration

Nhóm	Tham số	Giá trị	Mô tả
Voxel Grid	VOXEL_RES	0.1 m	Kích thước mỗi voxel
Voxel Grid	MAX_VOXELS	500,000	Giới hạn tổng số voxels
Occupancy	PROB_HIT	0.8	Tăng log-odds khi điểm LiDAR rơi vào voxel
Occupancy	PROB_MISS	-0.3	Giảm log-odds khi tia DDA xuyên qua voxel đã tồn tại
Occupancy	OCC_THRESH	0.6	Ngưỡng xác nhận trạng thái occupied
Occupancy	CLAMP_MIN	-1.2	Giới hạn dưới log-odds
Occupancy	CLAMP_MAX	1.6	Giới hạn trên log-odds
Eviction	MAX_FRAME_AGE	600 frames	Evict voxel sau 600 frame không xuất hiện
Raycasting	MAX_RAY_LENGTH	20.0 m	Giới hạn chiều dài tia DDA tối đa, tia dài hơn bị cắt ngắn trước khi duyệt
Timeout	UPDATE_TIMEOUT_US	50,000	Hard deadline 50ms cho update()
Pool	POOL_SIZE	510,000	Tổng slots pre-allocated

ikd-Tree	α_{bal}	0.6	Tiêu chí cân bằng
ikd-Tree	α_{del}	0.5	Tiêu chí xóa
ikd-Tree	N_max	1,500	Ngưỡng re-build song song
ikd-Tree	filter_size_map	0.5 m	Độ phân giải downsampling cho mỗi voxel
IEKF	NUM_MATCH_POINTS	5	Số lân cận gần nhất (5-NN) cho plane fitting
IEKF	NUM_MAX_ITERATIONS	4	Số vòng lặp tối đa của iterated update



2-Layer LiDAR-Inertial SLAM Architecture

ID:
Lần ban hành: 01
Ngày ban hành:
Trang: 52/53

X. Conclusion

Bài báo trình bày kiến trúc **2 lớp LiDAR-Inertial** được thiết kế chuyên biệt cho bài toán *Obstacle Avoidance* và *Path Planning* của UAV tự hành, không phải cho mục đích chính việc xây dựng *Global Map*.

Layer 1 (Fusion Engine) sử dụng FAST-LIO2 với IEKF ghép cặp chặt LiDAR-IMU và ikd-Tree để ước lượng pose 6-DOF và duy trì bản đồ điểm cục bộ.

Layer 2 (VoxMap Manager) nhận point cloud đã đăng ký và xây dựng bản đồ với cấu trúc spatial hashing, pool allocator, và cơ chế eviction chủ động để duy trì bản đồ vật cản cục bộ luôn cập nhật quanh UAV.